ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

Proof-of-Control: An Open Standard for Runtime Verifiability and Cryptographic Oversight in Autonomous AI Execution

Prove, rather than promise, what your agents did.

Jim Schwoebel[1,2] **Ken Huang**[2,3] **Tricia Wang**[2] **Michael Casey**[2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org` [3] Cloud Security Alliance[1]

Specification, reference implementation, and all artifacts: `https://github.com/AAI-Society/ov-poc-standard`

ABSTRACT

Suppose you let a piece of software act for you: read your records, call your tools, move your money. Afterward it tells you what it did. How would you check? Today, in nearly every deployment, you cannot. The only account of what happened is the system's own, and the party writing that account is the party you would be checking. We call this the *Verifiability Gap*, and this paper is about closing it. We describe Proof-of-Control, an open standard with six domains of verification covering 127 testable requirements; four tiers that grade evidence by *who you must trust to believe it*, with a hard line between the tiers where you still trust somebody and the tiers where you do not; and an architecture that intercepts every consequential action, decides it inside a protected environment, and emits a signed record chained to everything before it. We prove that this arrangement does *not* do what you would assume it does—an attester that signs "I evaluated this snapshot" says nothing about what the machine actually went on to do—and we show what you have to add to fix it. Then we built the thing and measured it. The software costs 201 µs per action, about one percent of our budget. Checking the path an agent took costs the same whether the path is ten steps or five thousand. Nine attacks work against the naive design and fail against ours—and three of them were defects in our own implementation, found by review rather than by design. We also measure what we did not expect: a monitor that watches the whole path refuses 42% of perfectly legitimate work unless you give it an explicit place to forget, which is exactly what people who study information flow have been saying for fifty years. Because an auditor should be able to check one action without downloading a million, we adopt Certificate Transparency's tree and show it turns a 123 MB download into a 544-byte proof. Because evidence has to be believable long after it is written, we measure surviving quantum computers: the hash chain already does, and FIPS 204 ML-DSA signatures cost a factor of 38 in size—a storage bill, not a cryptography problem, and one that halves again if you encode the evidence as CBOR instead of JSON. And because none of this is worth anything if two implementations disagree about which bytes they signed, we publish a machine-readable schema, a canonical form, and signed test vectors.

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript; the Initiative's working groups, board, and advisory board are credited in the Acknowledgments. Additional co-authors will be added only upon explicit consent and substantive contribution.

1 The Problem

A few years ago software did what you told it, step by step, and if you wanted to know what it had done you read the log. Now we have agents. You give one an objective in a sentence, and it decides for itself what to do: which database to query, which API to call, what to try next when the first thing fails, which sub-agent to hand a piece of the job to. It does this at machine speed, and it does not do the same thing twice [1, 2].

That is genuinely useful. It also breaks something we have quietly relied on for a long time, which is the ability to find out afterward what happened.

Here is the situation exactly. An agent reads a customer record, calls three tools, and issues a refund. It reports success. Did it open only the records it was allowed to open, or did it look at others while it was in there? Did it stay inside the authority you gave it, or did something it read along the way talk it into doing more? You cannot tell. There is a log, but the machine that performed the actions is the machine that wrote the log, and if that machine is compromised—or if the company running it would simply rather you did not know—the log says whatever they want it to say. An agent reporting that it behaved is not evidence that it behaved.

We call this the *Verifiability Gap*: the absence of independent evidence of what an AI system did [3]. It is not a policy gap. Nobody is confused about what the agent *should* do. It is an evidence gap—and those are different problems with different fixes.

The reason it matters more every year is arithmetic. As machines get cheaper at doing things, they do more things, and the number of actions somebody ought to check grows with them. But the checking is still done by people, and there are the same number of people as before. Catalini, Hui and Wu work this out carefully [4]: the cost of doing falls toward zero while the cost of verifying is pinned to human attention. So verification becomes the scarce thing. If you want to check machine work at machine scale, the checking has to be done by machines too, and the evidence has to be the kind a machine can check.

What we did. We wrote a standard, we analyzed it, we found holes in our own design—two by proving theorems and a third by building the code— we fixed them, and then we measured what the whole thing costs. Specifically:

1. **The standard** (Section 4): six domains of verification with 127 testable requirements, four evidence properties, four tiers that grade evidence by how much trust it demands, and three stages of how thoroughly a claim was checked.
2. **The architecture** (Section 5): interception of every consequential action, decided inside a hardware-protected environment, under the IETF attestation architecture [5], emitting signed tokens [6] chained together in the manner of tamper-evident logs [7, 8, 9].
3. **The analysis** (Sections 6–7): a precise statement of what an attacker can do, four properties we want, and proofs. Two of the four do *not* follow from the obvious design. We show exactly what has to be added.
4. **The comparison** (Section 8): how much of this eight existing frameworks already ask for, coded requirement by requirement, with an honest account of what that measurement can and cannot tell you—including a *cross-model audit* of our own coding (Section 8.2), because the party that wrote the requirements should not be the only party deciding whether anyone else already covers them. The procedure and its acceptance rule are reported in full, and we recommend it for any coverage mapping where the author and the coder are the same.

5. **The interoperable claim set** (Section 5.6): a machine-readable schema in two renderings, a canonical serialization that fixes exactly which bytes a signature covers, and signed test vectors including ten negative cases. Without these, two conforming implementations produce different digests for the same action and the central binding property fails silently.
6. **The implementation** (Section 9): working code, an attack harness that runs eleven attacks with and without each defense, measurements of what it all costs, what it takes to make the evidence outlive the signature scheme that protects it, and—in Section 9.11—logarithmic proofs that let an auditor check one action out of millions without downloading the rest.

One finding we did not go looking for. Building those proofs broke our own verifier. An operator holding the signing key can rewrite a record in the middle of the log, recompute every link after it, and re-sign the lot; the result replays perfectly, because nothing about it is internally inconsistent. Our verifier compared the published step *count* and never the published *root*, so it accepted the forgery. That is attack A9, and the fix is now requirement C7.3.5. It is worth noting how it was found: not by proving anything, but by implementing a performance optimization carefully enough to have to say what a published root was supposed to guarantee.

One thing we could not do. We did not have a machine with a real hardware enclave, so in our implementation the protected environment is a software stand-in: the signing key is never handed out, the "measurement" is a digest of the policy code, and every step of the protocol runs for real. But hardware isolation is not there. That means our timings leave out the cost of crossing into a real enclave, and the true number will be larger. We say so rather than guessing at a number we did not measure.

2 Why the Obvious Answers Don't Work

Before describing what we did, it is worth being clear about why the things people already do are not enough. Not because they are bad—they are useful—but because each one answers a different question.

2.1 The path is the problem

Old software followed a decision tree you could read in advance. An agent does not. Given one objective it invents its own sequence of steps, reads whatever comes back, and changes its mind. So you cannot check compliance before it runs; there is nothing fixed to check.

Worse, you cannot check it one step at a time either, and this is the part people miss. Consider two actions. First, the agent reads a confidential customer record—which it is allowed to do; that is its job. Second, it makes an outbound network call—which it is also allowed to do; it has to talk to partners. Look at either action alone and you see nothing wrong. Look at them in sequence and you are watching data walk out the door.

This is not a new observation. It is the oldest observation in the field of information flow [10, 11]. What is new is the setting: benchmarks for tool-using agents show that defenses which work fine on isolated prompts fall apart over multi-step sequences [12, 13], that content retrieved from the world hijacks agents reliably [14, 15], and that harm usually arrives as a series of individually reasonable calls [16]. We return to the information-flow connection in Section 6.1, because it turns out to predict our most important measurement.

2.2 Three things people do, and what each one misses

Telling the model to behave. You write a system prompt: do not do X, always do Y. The trouble is that the instruction lives in the same stream of text as everything the agent reads—user input, retrieved documents, tool output. There is no wall between "what I was told by my owner" and "what I read on a web page." So a web page can tell it something else. And even setting attacks aside, a prompt is a suggestion with statistical force, not a barrier. If the model steps over the line, nothing stops it. There is no line; there is only a preference.

Permissions. Give the agent credentials for exactly the endpoints it needs. This is real and you should do it. But permissions are blind to context and to history [17]. An agent that legitimately needs to read the customer database, and legitimately needs to send email, has everything it needs to exfiltrate the customer database by email, and every individual call is authorized. That is the path problem again, and a permission system cannot see it because it looks at one call at a time.

Guardrails inside the application. Put the checks in the agent's own code. Now ask: what happens when the agent is running generated code, or has a shell tool? The checks are running with the same authority as the thing they are checking. A process can skip its own checks. And the log it writes is its own log [18]. This is the deep problem with all three: *the thing being checked is doing the checking.*

2.3 Why looking afterward is too late

The usual backstop is the audit: collect the logs, look later. Two things have gone wrong with that.

First, the time available has collapsed. An instruction hidden in a web page can redirect an agent instantly, and the money moves before anyone reads anything [2]. Afterward is measured in weeks; the event was measured in milliseconds.

Second—and this is the fatal one—ordinary logs are not evidence. They are a story the system tells about itself. Anyone who controls the machine can edit them, add to them, or quietly leave things out. So the audit rests on trusting the operator, which is precisely the thing you were trying not to have to do.

What you want instead is evidence made *by the mechanism, as the action happens*, anchored to something the operator cannot reach [19, 20].

3 What Others Have Built

Governing agents at runtime. The Agent Control Specification defines where to put the hooks and how to carry policy between systems [21, 22], and CSA's AARM defines what to do at the boundary: approve, modify, defer, deny [23]. That is the enforcement half, and it is necessary. But the record it leaves is still the operator's record. Enforcement decides what an agent may do; we are after evidence of what it did.

Attestation. The IETF's remote attestation architecture [5, 24] gives us the vocabulary and the roles, entity attestation tokens give us the format [6, 25], there is work on using several verifiers at once [26], and hardware enclaves plus verifiable-compute tooling give us somewhere to stand that the host cannot reach into [19, 20]. These prove things about *platforms and code.* We are extending them to say something about *what an agent did.*

Tamper-evident logs. Chaining records so that changing one breaks the rest goes back to Merkle [7] and was worked out properly for logging by Crosby and Wallach [8], then deployed at scale in Certificate Transparency [9, 27]. That literature also tells us what such logs *cannot* do on their own: an operator can show different histories to different people, or simply stop showing you the recent part. Which is why Certificate Transparency needs gossip and consistency proofs [28, 29], and why systems like CHAINIAC bring in witnesses [30]. We use those results in Section 7; as far as we can tell, nobody has applied them to agent evidence.

Reference monitors. The idea that every security-relevant operation must go through a mechanism that is tamper-proof, always invoked, and small enough to verify is Anderson's [31], and "always invoked" is Saltzer and Schroeder's complete mediation [32]. Section 5.3 is really just this old requirement, applied carefully, and finding that the obvious agent design fails it.

What enclaves cannot promise. Enclaves have been attacked, by architectural analysis [33] and by transient-execution tricks [34], and their trust ultimately roots in a chip vendor. We therefore write those down as assumptions instead of pretending they are free.

Standards and regulation. NIST's risk framework [35] and its generative profile [36], ISO/IEC 42001 [37] and 23894 [38], SOC 2 [39], the EU AI Act [40], OWASP AISVS [41], MITRE ATLAS [42], and zero-trust architecture [17] all govern neighboring ground. Section 8 measures exactly how much of what we ask for they already ask for, using the coding method HAARF developed for healthcare agents [43].

Identity. Decentralized identifiers [44], verifiable credentials [45], and supply-chain attestation formats [46, 47] give us the pieces for saying which agent acted on whose behalf, and where the model came from.

4 The Standard

Here is the whole idea in one sentence: *for every control you claim, the mechanism that enforces it must produce evidence, while it runs, that it held; you must say how independently that evidence can be checked; and you must say what you are still asking people to take on faith.* A system has Proof-of-Control only when its evidence reaches the top two tiers [3]. We use MUST and SHOULD in the RFC 2119 sense [48].

4.1 The six things worth having evidence about

The requirements—127 of them, each phrased as something you can go and check—are grouped into six domains:

1. **Provenance**: which model actually ran, and where it and its inputs came from, tied together with signed manifests and hash links [46, 47].
2. **Privacy**: what data was read and written—proved without handing over the data you are protecting, which is the awkward part.
3. **Portability**: what crossed which boundary, between companies, jurisdictions, or clouds, with the evidence chain surviving the trip.

4. **Authorization**: what authority was granted, whether each action stayed inside it, and whether delegation was valid—judged over the *path*, not one action at a time.
5. **Identity**: which agent, on whose behalf [44, 45].
6. **Security**: whether the environment was intact, whether the controls held, which tools were called, and who holds the keys.

Four more chapters cut across all six: how evidence is generated and what properties it must have, how it is graded, where in the stack it applies (we use the seven MAESTRO layers [49]), and how a claim is checked and what residual trust must be disclosed.

4.2 Four tiers, and the line that matters

The grading question is not "did you use cryptography?" Cryptography is everywhere and proves nothing by itself. The question is: *who do I have to trust to believe this?* (Figure 1.)

At Tier 1 you trust the operator: they say so. At Tier 2 you trust somebody else—an auditor, a certificate authority, a chip vendor. Note that a signed log is Tier 2: the signature proves the log was not edited after it was written, and proves exactly nothing about whether it was true when it was written. At Tier 3, there is nobody left to trust; anyone can check the evidence with published tools. At Tier 4, checking is not optional—the system cannot act unless the evidence exists.

The line falls between Tier 2 and Tier 3. Below it you have authenticated paperwork. Above it you have evidence. We insist on a hard line rather than a smooth scale for a practical reason: a buyer can ask a yes-or-no question. "Does your system have Proof-of-Control?" is a question a procurement officer can put in a contract. "How far along the trust-minimization spectrum are you?" is not. Every certification that ever built a market did this—FIPS validated or not, PCI compliant or not.

Separately from the evidence, we grade how carefully the *claim* was checked: the operator says so (Self-Declared), an accredited assessor checked (Third-Party Assessed), or it is checked continuously as the system runs (Continuously Monitored). And because you cannot govern what you have not found, the declared inventory of agents must be reconciled against what automatic discovery actually finds.

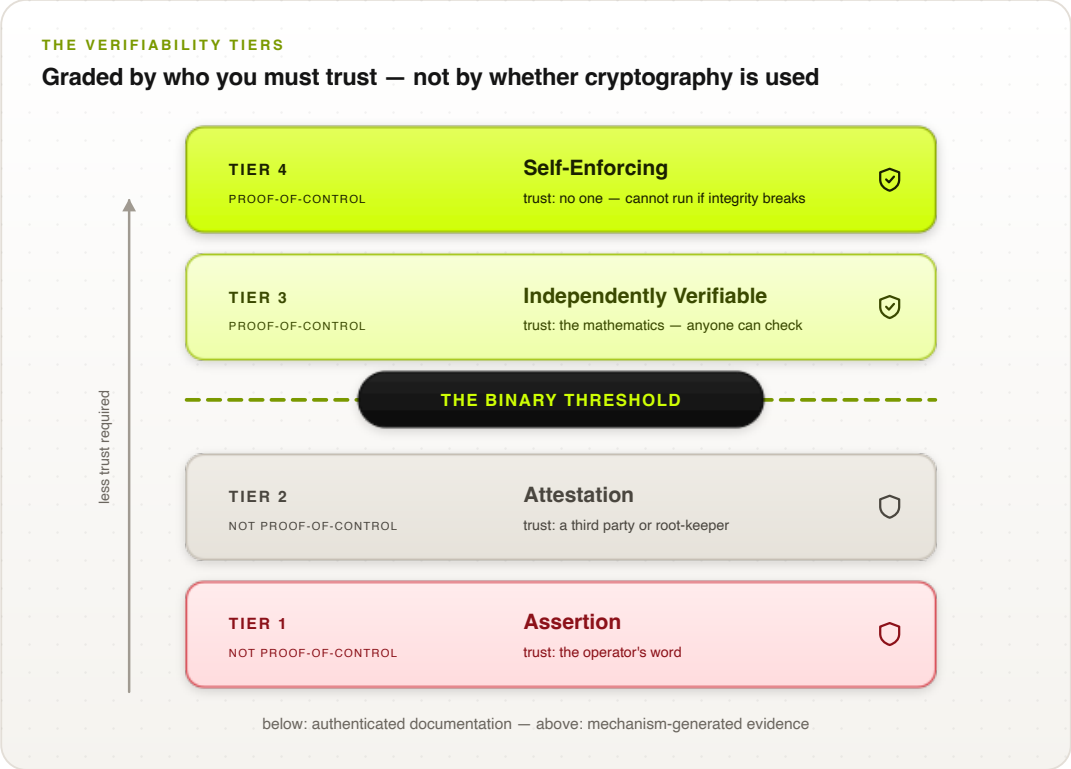

Figure 1: The four tiers, graded by who you must trust. The hard line sits between Tier 2 and Tier 3: below it, authenticated paperwork; above it, evidence a stranger can check. Requirement levels 1–4 line up one-for-one with the tiers.

5 How It Works

The architecture is easier to describe than it sounds (Figure 2). Every time the agent is about to do something that matters, we stop it. We write down exactly what it is proposing to do, in a form that hashes to a fixed value. We hand that description to a protected environment the host cannot reach into. That environment checks it against the policy, decides, adds a link to a chain of everything decided so far, signs the result, and hands back a token. Only then does the action go out. If the evidence cannot be written, the action does not happen.

5.1 Where we stop the agent

There are eight places, taken from the Agent Control Specification [21, 22]: when a task starts; when context is assembled (retrieval, memory); when a plan is formed; before a tool call; after a tool result comes back; when memory is written; when a sub-task is handed to another agent; and when the task finishes. At each one we build the snapshot and get back one of four answers: ALLOW, DENY, MODIFY (sanitize it and proceed), or ESCALATE (ask a human).

5.2 The roles, in the standard vocabulary

Mapping onto the attestation architecture [5]: the machine running the agent and the hooks is the *Attester*; the layer holding the runtime, the model context, and the proposed action is the *Target Environment*; the enclave that measures the policy code, checks the bundle signature, evaluates,

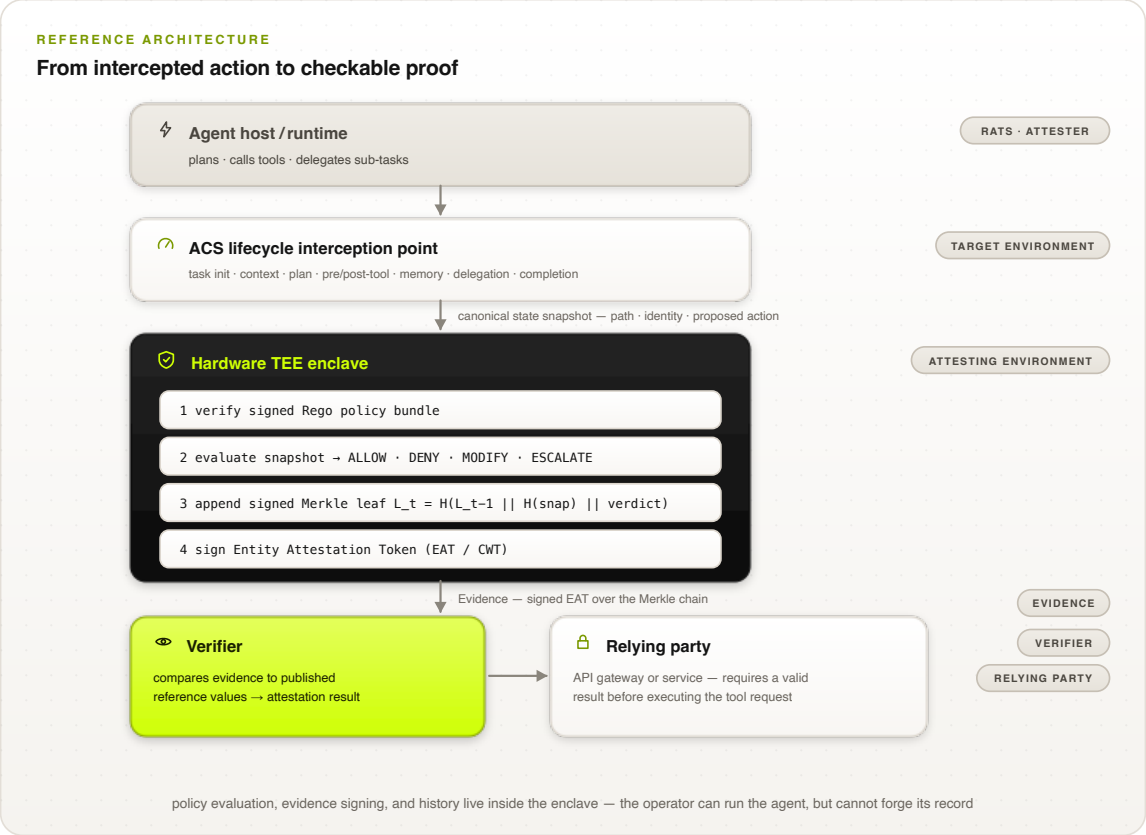

Figure 2: The pipeline, and how it maps onto the IETF attestation roles. The agent host is the Attester; the enclave is the Attesting Environment; the token is Evidence; a Verifier checks it against published reference values before a Relying Party will act.

and signs is the *Attesting Environment*; the signed token is *Evidence* [6, 25]; whoever compares that against published reference values is a *Verifier*; and whoever refuses to act without a good answer is a *Relying Party*. Chains of sub-agents are handled with several verifiers [26].

5.3 The mistake we almost made

Now here is the interesting part, and it is the part we got wrong at first.

The enclave signs a statement. It is worth reading that statement carefully, because it is narrower than it looks. What it says is: *given this description, the authorized policy said yes.* That is all. It does not say the description was true. It does not say the machine then went and did that thing.

So suppose the host is compromised. The host is what builds the description, and the host is also what holds the credentials and the network connection. It can hand the enclave a description of something harmless, collect a perfectly genuine ALLOW token, and then go do something else entirely. Every signature checks out. The evidence is impeccable. And it is a lie.

We call this *snapshot substitution*. It is not a clever attack; it is what happens if you forget Anderson's rule that the monitor has to be *always invoked*, not merely present [31, 32]. You have built a tamper-evident diary when you thought you were building a reference monitor.

The fix is that the channel through which the effect actually happens has to be inside the same trust boundary as the decision. Concretely, one of these must be true, and the claim has to say

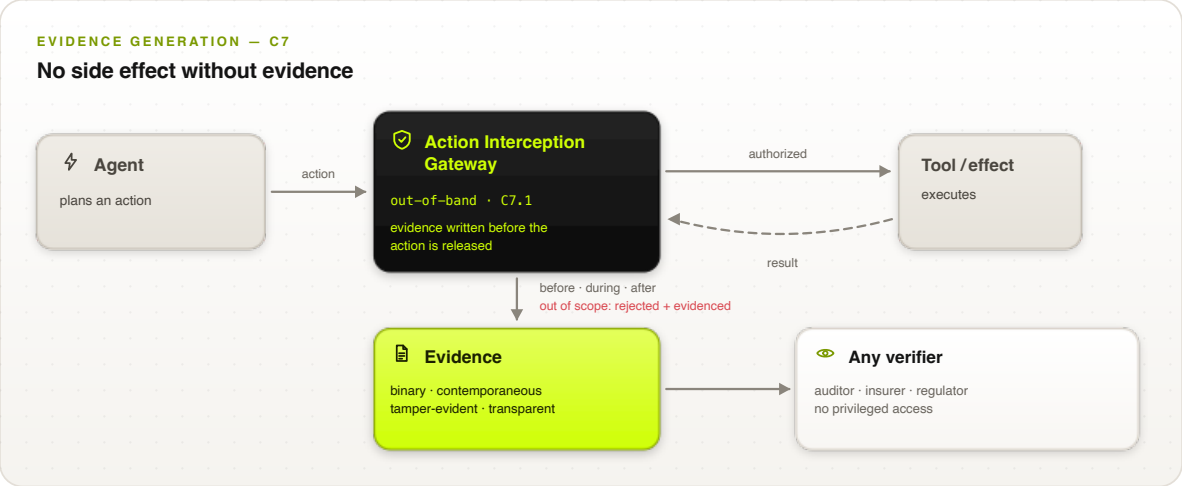

Figure 3: The same boundary, seen from the evidence side. Nothing gets out without a record, including the things that were refused.

which:

1. **The enclave makes the call itself.** It holds the credentials and the connection; the host has no path to the endpoint.
2. **The enclave issues a ticket.** A single-use capability cryptographically tied to the description and the target, which the far end checks before doing anything. A request without a matching ticket is refused where it lands.
3. **The path is attested.** Traffic can only leave through an enforcement point that is itself attested and only admits requests carrying matching evidence.

Option 2 is the only one that survives a fully compromised host without putting the network stack inside the enclave; options 1 and 3 shrink the problem rather than removing it. If you do none of these, you may still claim the lower tiers—but you must not claim your evidence describes what your system did, because it does not.

5.4 Chaining the records

Each step makes a link:

$$L_t = H(L_{t-1} \,\|\, H(\sigma_t^{\text{snap}}) \,\|\, d_t)\,, \qquad L_0 = H(\text{seed}_{\text{init}}), \tag{1}$$

where σ_t^{snap} is the description, d_t the decision, and H a hash nobody knows how to find collisions in. The enclave signs each link. Numbering the steps means a missing one leaves a hole. Periodically the running total is published somewhere public, off the critical path.

That much is standard [7, 8]. It is also not enough, and Section 7 shows exactly how it fails—an operator can stop publishing, or publish two different histories to two different people. Fixing those needs bounded publishing intervals and people comparing notes, which is what Certificate Transparency figured out [9, 28, 29].

5.5 What a record actually contains

Listing 1 is a real token, from the tool-call hook, for a payment API request. It is an entity attestation token [6] with our claims inside it. Table 1 says what every field means.

Three choices in there are worth pointing out. First, *the data never leaves*: the token carries a hash of the description, not the description, so anyone can later confirm what was evaluated by hashing it, and the evidence store never becomes a second copy of your customer database. Second, *history is load-bearing*: the step number and the running total tie this record to every record before it, so quietly dropping an inconvenient step breaks arithmetic rather than merely being impolite. Third, *the rules are pinned*: the bundle hash names the exact policy that produced this answer, so "which rules were in force?" is something you check, not something you reconstruct from a change log and hope.

How you check one. It is mechanical, and worth walking through because there is nothing clever in it. (1) Check the signature against the platform's attestation roots. (2) Compare the enclave measurement to the published golden value—this is what tells you the code that decided was the code you think decided. (3) Check the policy bundle hash against the registry. (4) Check the nonce is fresh, so this is not an old token being waved at you again. (5) Check the step numbers run consecutively and the running totals agree with the other tokens you have seen. (6) Only now believe the verdict. Every one of those steps uses published material. You never ask the operator for anything. That is the entire point.

5.6 Making two implementations agree

Table 1 is prose in a grid. It is enough for a reader and not enough for an implementer, and the gap between those two is where a security property quietly dies.

Here is how. Theorem 1 works because the relying party recomputes a digest over the request it has been asked to perform and compares it to the digest the enclave committed to. Now suppose your implementation serializes an action with the keys in the order they were inserted, and mine sorts them. Same action, different bytes, different digest. Every signature still verifies. Nothing throws an exception. The comparison just fails, for no reason anyone can see from the outside—and the natural thing for a tired engineer to do at that point is loosen the comparison until the two systems interoperate. Which removes the very thing the comparison was there for.

So the claim set now comes with three artifacts rather than a table. There is a CDDL definition, which is the form an IETF attestation profile is actually written in, plus a JSON Schema for deployments that use JWTs. There is a document that fixes the canonical form: key ordering, string escaping, digest case, what "absent" means, and exactly which bytes a signature covers. And there are signed test vectors, which are the part that does the real work.

We made two decisions worth repeating. Digests are algorithm-tagged— `sha-256:9f86d0…`, never a bare hex string—and a verifier handed an untagged digest must refuse it rather than assume SHA-256. Assuming the algorithm is exactly how a hash migration goes wrong, and it goes wrong silently. And the claim set contains no floating-point numbers at all: `iat` is integer seconds, ordering comes from the step index. Number formatting is the single hardest part of canonicalization, and the cheapest way to get it right is to not have any numbers that need formatting.

Ten of the vectors are negative—each breaks exactly one rule, so that rejecting it demonstrates one specific check exists. A vector rejected for the *wrong* reason is not a pass: a validator that chokes on the duplicate-key vector because it could not parse the file has not shown that it detects duplicate keys.

That duplicate-key vector is the one we would point at if we could only point at one. It is a single file that a last-wins parser reads as ALLOW and a first-wins parser reads as DENY. If the relying party and the auditor make different choices there—and both choices are common, and neither is wrong in any documentation—an operator can show each of them the answer it wants

```
{
  "iss": "https://verifier.trusted-attestation.org",
  "iat": 1773532800,
  "nonce": "0x8f93e2a110c49b",
  "eat_profile": "https://standards.org/poc/v1",
  "poc_claims": {
    "agent_id": "did:web:enterprise.com:agents:finance-agent-04",
    "initiating_user": "user:auth0|84f12a9e",
    "agbom_digest": "sha256:7f8a...5f6a",
    "interception_point": "PRE_CALL_TOOL_INVOCATION",
    "step_index": 3,
    "merkle_root": "0x2c3d...2c3d",
    "policy_bundle_hash": "sha256:e3b0...b855",
    "target_resource": "api.bank.com/v1/payments",
    "canonical_snapshot_hash": "sha256:a4b3...a1b2",
    "verdict": "ALLOW"
  },
  "submods": {
    "hardware_attestation": {
      "platform": "INTEL_TDX",
      "mr_config": "0x1a2b3c4d5e6f...",
      "attestation_signature": "0x30440220..."
    }
  }
}
```


Claim	Type	Meaning
`iss` / `iat`	EAT	Token issuer and issuance timestamp.
`nonce`	EAT	Relying-party challenge; proves freshness and prevents replay of an old token for a new action.
`eat_profile`	EAT	The Proof-of-Control EAT profile (and version) this token conforms to.
`agent_id`	PoC	Decentralized identifier of the agent instance (C5); resolvable to its public keys.
`initiating_user`	PoC	The principal on whose authority the action runs—the human-to-agent binding (C5.1).
`agbom_digest`	PoC	Digest of the Agent Bill of Materials: the build, tools, and model manifest the agent runs from (C1).
`interception_point`	PoC	Which lifecycle hook produced this token (one of the eight in §5).
`step_index`	PoC	Monotonic position in the execution path; gaps expose omitted steps.
`chain_head`	PoC	Running hash link over all steps so far; what a verifier replaying the whole history recomputes.
`merkle_root`	PoC	RFC 6962 tree head over the same leaves; what an *inclusion proof* reconstructs (§9.11).
`tree_size`	PoC	Number of leaves the root covers. An inclusion proof means nothing without it.
`policy_bundle_hash`	PoC	Digest of the signed Rego bundle that was evaluated—pins the exact policy in force.
`target_resource`	PoC	The resource the proposed action addressed (tool endpoint, table, device).
`canonical_snapshot_hash`	PoC	Commitment to the full canonical state snapshot that was evaluated; discloses nothing by itself.
`verdict`	PoC	The binding decision: ALLOW, DENY, MODIFY, or ESCALATE.
`platform` / `mr_config`	HW	TEE platform and enclave measurement; compared against published reference values to prove the authorized policy engine ran unmodified.
`attestation_signature`	HW	Hardware-rooted signature over the token by the enclave-bound key.

Table 1: The Evidence claim set. "EAT" rows are standard Entity Attestation Token claims [6]; "PoC" rows are the Proof-of-Control profile; "HW" rows form the hardware-attestation submodule. Every field here has a machine-readable definition, a canonical serialization, and signed test vectors—see §5.6, because a table like this one is not enough to make two implementations agree.

and neither can tell. That is not a parsing preference. That is one artifact with two meanings, which is the whole ballgame.

The reference implementation's own output is run through the validator in its test suite, so the schema and the code cannot drift apart without something going red. And because the CDDL is a real definition rather than an aspiration, we implemented the CBOR rendering too and round-tripped every vector through it. The two carry identical claim sets, and CBOR is 48 % smaller, because digests and signatures travel as bytes instead of as hexadecimal text. That turns out to matter more than it sounds: as §9.7 works out, switching encoding saves more storage than post-quantum signatures cost.

6 Writing It Down Precisely

Words like "the policy decides" are fine until two people disagree about what they mean, so here it is in symbols.

Let $\mathcal{A}$ be the actions an agent can propose, $\mathcal{I}$ the identities it can act under, and $\mathcal{S}$ the states of the machine. The steps taken so far form a path $p_t = \langle a_0, \ldots, a_{t-1} \rangle \in \mathcal{A}^*$. A policy is a function that looks at all four things and gives one of four answers:

$$\Pi : \mathcal{A} \times \mathcal{I} \times \mathcal{S} \times \mathcal{A}^* \longrightarrow \mathcal{D}, \quad \mathcal{D} = \{\textsc{allow},\ \textsc{deny},\ \textsc{modify}(a'),\ \textsc{escalate}(h)\}, \tag{2}$$

where a' is a cleaned-up version of the action and h says which human to ask. This function has to run inside the enclave, and it produces, for each step, a proof

$$\pi_t = \mathrm{Sign}_{k_{\mathrm{AE}}}(H(\sigma_t^{\mathrm{snap}}) \parallel d_t \parallel t \parallel L_t)\,, \tag{3}$$

with k_{AE} a key nothing outside the enclave can use.

The fourth argument is the one that matters. Because Π sees the path, the read-then-send trick from Section 2.1 is inside what the policy can reason about, instead of being invisible to it.

And here is what we are careful *not* to claim. We verify facts about execution: which model ran, on what input, under whose authority, producing what effect. We do not verify that the output was correct, or fair, or wise, or that the boundaries somebody chose were the right boundaries. Those are human judgments and they stay human judgments [37]. It is easy to let a system like this drift into implying more than it proves, and we would rather draw the line ourselves than have someone else draw it for us later.

6.1 This is information flow, and we should say so

Watching a path to catch read-then-send is not a new idea. It is information flow control, which has been studied since the 1970s: Denning's lattice model [10, 50], noninterference as Goguen and Meseguer defined it [11], and decades of systems that carry labels around at the language and operating-system level [51, 52, 53, 54, 55].

So rather than pretend we invented something, let us be exact about the differences, because they are real:

- **How fine-grained.** Classical systems label variables and processes inside a runtime they trust. We label *actions at a boundary*, because the runtime here is a language model whose insides we specifically decline to trust. Ours is coarser. It is also checkable from outside, which theirs is not.

- **What we are after.** Those systems *enforce* a flow property. We enforce a policy *and hand you evidence* that we did, which you can check without having watched. The evidence is the contribution; the enforcement is an old idea done at a new place.
- **What we promise.** No noninterference theorem. An agent whose permitted actions are themselves harmful, or which leaks through a channel we never intercept—timing, or the content of a permitted message—is outside what we claim.
- **What we inherit.** Label creep and the declassification problem [53]. A monitor that accumulates restrictions along a path eventually refuses to let anyone do anything. The literature has been saying this for fifty years. We did not believe it hard enough until we measured it in Section 9.4, where it cost us 42% of legitimate work.

6.2 Keeping the cost from growing

Written as above, Π takes the whole path as an argument, and the path grows forever. If evaluating step t costs something proportional to t, then a long-running agent eventually stalls. That is not acceptable, so we require policies to be expressible over a *bounded summary*: a state ϕ_t of size at most B, updated by folding in one step at a time,

$$\phi_t = \mathsf{upd}(\phi_{t-1}, a_{t-1}, d_{t-1}), \qquad \phi_0 = \phi_{\text{init}}, \tag{4}$$

so the implemented policy sees $\tilde{\Pi}(a, \iota, s, \phi_t)$ instead of the whole history. The summary is doing the same job an IFC label does: it carries the few facts the rules need—the highest classification read so far, how much budget is left, whether anything has been declassified—and forgets the rest.

Two things follow. The cost per step is $O(|\tilde{\Pi}| + B)$, flat no matter how long the agent runs, and we commit $H(\phi_t)$ in the evidence so a verifier can confirm the policy was evaluated against the right accumulated state. But we also give something up: rules that need unbounded history ("never contact an endpoint you did not see in the first k steps" for arbitrary k) cannot be expressed exactly and must be approximated. That is a real restriction. We would rather state it than quietly leave the design unimplementable. Section 9 shows the flat cost is not just theory.

7 What an Attacker Can Do

7.1 Who we are up against

We assume an attacker $\mathcal{A}$ who can run in polynomial time and who (i) owns the operating system, the container, the orchestration, and the network—start, stop, reorder, replay, drop whatever it likes; (ii) *is the operator*, and wants to look compliant. That second one is unusual and it is the whole point: in an evidence system, the party writing the record is the party being examined. Also (iii) can put any text it likes in front of the agent [14, 15], and (iv) can string together permitted actions in any order.

What we assume it cannot do: **(A1)** forge signatures or find hash collisions; **(B1)** get inside the enclave—and we should be honest that this assumption has been dented in practice [33, 34]; **(C1)** get the chip vendor to vouch for a lie, which is a *person* we are trusting, and exactly why vendor-rooted evidence alone does not get above Tier 2; and **(D1)** cut every verifier off from every other verifier.

7.2 Four things we want, and which ones we get

P1 (The verdict is real). The decision in the token came from the authorized rules applied to the description in the token. *We get this* under (A1), (B1), (C1): the measurement pins the code, the bundle hash pins the rules, and the signature ties the verdict to the description.

P2 (The evidence describes what happened). *We do not get this* from attestation. This is snapshot substitution from Section 5.3: without mediating the effect channel, $\mathcal{A}$ produces a true statement about a fictional action. P1 holds while P2 fails, which is the worst kind of failure—a perfect record of the wrong thing.

P3 (Nothing was changed or dropped). *We get this within a stretch of history you actually see*: chaining catches edits, numbering catches holes. *We do not get it at the end*: if the operator simply stops showing you the last few steps, nothing in the chain complains, because a chain cannot tell you about a link you were never handed. Publishing the running total on a bounded schedule fixes this, and shrinks the blind spot to the publishing interval Δ.

P4 (Everyone sees the same history). *We do not get this* from signing and publishing alone. The operator can keep two sets of books and show each verifier a different one; both are internally perfect. This is the split-view attack, well known in transparency systems [9, 28, 29]. The fix is that verifiers must compare notes—gossip, a quorum of witnesses [30], or a ledger that only admits one history.

Table 2 maps the threats to which property they attack.

Threat	Attacks	What stops it, and what is left over
Operator turns off enforcement or edits records	P1, P3	Rules and keys live in the enclave; changing the rules changes the measurement and verifiers stop believing it. Left over: (B1), (C1).
Snapshot substitution	P2	Mediate the effect channel (Section 5.3). Left over: options 1 and 3 still trust the host not to fiddle with transport configuration.
Stop publishing recent history	P3	Bounded publishing interval Δ; a missing publication is itself an alarm. Left over: whatever happened in the last Δ.
Two sets of books	P4	Gossip, witnesses, or consensus. Left over: witnesses who collude.
Injected instructions from the world	P1 (indirectly)	The enclave applies fixed rules and does not read persuasion, so an injected instruction cannot authorize an out-of-scope action. Left over: harmful actions that are inside scope—a safety problem, not a verifiability one [14, 16].
Stringing permitted actions together	P1	The policy sees the path (Section 6). Left over: the cost in refused legitimate work, measured in Section 9.4.
Sub-agents used as cutouts	P1–P4	Constraints are inherited; sub-agent evidence checked by several verifiers [26].
Breaking the enclave	P1–P4	Out of scope by (B1)—disclosed, not solved, and bounded by re-attestation and key rotation.

Table 2: Threats against properties. P2 and P4 are the ones you do *not* get for free, which is why the standard has extra requirements for them.

7.3 Proving the two hard ones

P1 and P3 are ordinary signature-and-chain arguments. P2 and P4 are the interesting ones, so we state them properly.

Definition 1 (Evidenced execution)**.** *Fix an agent ι and a relying party R. An effect e carried out at R is evidenced if some token τ that a verifier accepts commits to a description digest $H(\sigma^{\text{snap}})$, a target r, and the verdict* ALLOW*, where σ^{snap} describes exactly e and r is where e happened.*

Definition 2 (The execution-fidelity game $\mathsf{Exp}^{\text{fid}}$)**.** *$\mathcal{A}$ has the powers listed above and may talk to the enclave $\mathcal{E}$ as much as it likes, collecting tokens and tickets. It wins if a compliant relying party R carries out an effect e that is not evidenced in the sense of Definition 1.*

In plain words: the attacker wins if it can get something done that the paperwork does not describe.

Theorem 1 (Tickets give you execution fidelity)**.** *Suppose for each* ALLOW *the enclave issues $c = \text{Sign}_{k_{\mathcal{E}}}(H(\sigma^{\text{snap}}) \,\|\, r \,\|\, \text{nonce})$, and a compliant R acts only when the request carries a c that (i) verifies under an enclave key whose measurement matches the published value, (ii) names R's own resource, (iii) carries an unused nonce, and (iv) matches the request itself against $H(\sigma^{\text{snap}})$. Then for any polynomial-time $\mathcal{A}$,*

$$\Pr[\mathcal{A} \textit{ wins } \mathsf{Exp}^{\text{fid}}] \;\le\; \mathbf{Adv}^{\text{EUF-CMA}}_{\text{Sig}}(\mathcal{A}) \;+\; \mathbf{Adv}^{\text{coll}}_{H}(\mathcal{A}),$$

under (A1), (B1), (C1).

Proof sketch. Say $\mathcal{A}$ wins: R did e, and no token describes e at r. By check (i) the ticket verified under the enclave key, which by (B1) never leaves and by (C1) is tied to the attested measurement. So either $\mathcal{A}$ forged a signature—that case is bounded by $\mathbf{Adv}^{\text{EUF-CMA}}$—or the enclave really issued the ticket. If it really issued it, then by check (iv) the request that was carried out hashes to the digest inside the ticket. If the description had been of something other than e, two different descriptions would share a digest, which is bounded by $\mathbf{Adv}^{\text{coll}}_{H}$. Otherwise the description *is* of e; by check (ii) the target matches; and the enclave only issues tickets alongside an ALLOW token it also emits. So e is evidenced—contradiction. Check (iii) stops a ticket being reused for a second effect. □

Proposition 1 (Without the far-end check you lose immediately)**.** *If nothing at the point of effect ties the request to $H(\sigma^{\text{snap}})$, then an attacker owning the host wins with probability 1.*

Proof. Hand the enclave a description of something harmless, take the genuine ALLOW, then use the host's own credentials to do something else. No check anywhere is violated: the token is real and the enclave's statement is true. The effect is unevidenced, so the attacker wins every time. □

We want to be blunt about what Proposition 1 means, because it is the thing we would most like a reader to take away. It is not a corner case or a hardening tip. It says that an evidence architecture without effect mediation is broken at probability one while every signature verifies perfectly. That is why the requirement is a MUST.

Definition 3 (Equivocation)**.** *$\mathcal{A}$ equivocates if two verifiers accept histories for the same agent where neither is a prefix of the other.*

Theorem 2 (Two sets of books can be caught, and pinned on someone)**.** *Suppose each running total at index i is signed by the enclave, and two verifiers exchange (total, index) pairs—directly, through a witness quorum with at least one honest member, or via a single-history ledger. If $\mathcal{A}$ equivocates, then under (A1) and (D1) the honest parties end up jointly holding two validly signed totals at the same index with different values. That is a non-repudiable proof of equivocation, attributable to the enclave key. Detection probability is 1.*

Proof sketch. Two histories that are not prefixes of one another differ at some earliest index i. Both verifiers hold signed totals at i; when they compare, the values differ. Forging either reduces to signature forgery, ruled out by (A1). And the pair stands on its own—you do not have to believe either verifier's story about it. □

Notice what Theorem 2 does *not* say. Gossip does not prevent an operator from keeping two sets of books. It makes it impossible to keep them secret, and it produces a receipt with a name on it. That is the same guarantee Certificate Transparency provides [9, 29], and it is worth being precise about, because "we use gossip" sounds like prevention and is not.

7.4 Building Tier 4

Tier 4 says the system cannot act unless verification succeeds. That has to be a construction, not a promise, and it falls out of the ticket scheme. If the far end refuses any request without a valid ticket, then: when the enclave is not running, or its measurement does not match, no ticket exists, so nothing happens. The system fails closed *because of how it is built*, not because somebody left a flag set. And the decision to stop is not the operator's to make—which is the difference between this and a "halt" variable in a process the operator controls and can patch out. What is left over is the far end's own enforcement and the chip vendor (C1), both of which must appear in the disclosure.

This also settles a tension people notice: a TEE attestation on its own is Tier 2, because (C1) is a person you are trusting. You get to Tier 3 by adding independent anchoring, so nobody has to take the vendor's word about which code ran, and to Tier 4 by making valid evidence a precondition of acting. The tiers grade the shape of the trust, not the brand of the cryptography.

7.5 Will anyone actually do this?

Theorem 1 asks the far end to check tickets, and it is fair to ask whether that is realistic. Three things.

The mechanism is old. Action-bound tickets are capabilities, which have been around since the 1960s [56, 57], and macaroons already give you exactly this shape—bearer tokens with cryptographic caveats, checkable by the service without calling anyone [58], over ordinary HTTP headers. A check is a signature verification and a comparison, not a new industry.

You can start alone. Options 1 and 3 need nobody's cooperation: run egress from inside the enclave, or force it through an attested proxy. Inside one company, the API gateway is already under your control, so you can turn on ticket checking there first. Partners come next by contract—which is how SOC 2 requirements already propagate—and standardization last, which is what the attestation-token profile work is for.

Some endpoints will never do it. Arbitrary third-party services and legacy systems can only be covered by options 1 or 3, whose guarantee is weaker. We are not claiming a universal answer. We are claiming a property, three ways to get it with different amounts left over, and a rule that you have to say which one you used—so the person relying on your evidence can tell the difference.

8 How Much of This Do Existing Rules Already Ask For?

Before claiming a gap exists, you should check whether somebody already filled it. So we took all 127 requirements and, one at a time, asked of eight existing frameworks: does this framework already require this?

8.1 How we did it

Using the coding method HAARF worked out for healthcare agents [43], each requirement gets one of three marks against each framework. *Exact*: they ask for the same thing, at the same depth. *Partial*: they ask about the same topic, but without demanding evidence somebody outside the company can check, or not in as much detail. *None*: they do not address it. Coverage is (Exact + Partial) over 127. The sheet, the rubric, and the script are public, so you can re-derive the numbers or re-code a framework you think we got wrong.

8.2 Checking the coding with a different model

The requirements and the coding sheet were produced by the same party. That is a structural weakness in any coverage mapping: whoever decides that an external framework does not already cover a requirement is the party that wrote the requirement. The two-coder study the rubric calls for is the answer to it, and it has not been run.

What can be run cheaply is a cross-model audit. A model from a different vendor reviews every row—one independent pass per framework, without access to the original reasoning, instructed to look for miscodings in both directions and to name a specific clause for every proposal. The procedure matters more than the idea, because the cheap version of it is worthless.

The failure mode is fabrication. A model asked for a clause citation will produce a plausible-looking one, and a fabricated citation is worse than no citation because it survives casual review. Three controls address it.

An acceptance rule fixed in advance. A proposal is applied only if it can be verified *without* trusting the reviewer about a framework's contents; everything else is recorded rather than applied. Here that admitted 31 of 192. Twenty-four were Exact ratings on the conformance-statement requirements, settled by reading the requirement text: those describe an artifact this standard defines, so nothing external can be equivalent in scope and intent to it, and no knowledge of SOC 2 is needed to establish that. Seven cited MITRE ATLAS mitigation identifiers confirmed to exist. The remaining 161—including all 91 proposed PARTIAL-to-NONE downgrades—wait for a human with the source document open.

A check on the direction of the corrections. Applying every proposal would have removed about 64 covered rows and taken 9 to 15 points off most frameworks: the direction that widens the gap this standard exists to fill. A review that finds errors in only one direction is measuring the reviewer rather than the sheet. This one proposed 37 changes that *raised* external coverage, which is what makes its downgrades worth reading.

A factual spot-check. One claim was checkable against a published source—that the MITRE ATLAS coding drew on a superseded mitigation set. Current ATLAS includes AI Bill of Materials (AML.M0023), AI Telemetry Logging (M0024), Maintain AI Dataset Provenance (M0025), and

agent-specific permission mitigations (M0026–M0028). Coding them moved ATLAS coverage from 25% to 30%—upward, against the direction that would flatter this standard.

What it returned. 192 proposed changes and 96 citations that do not name a checkable clause. Three results are structural rather than row-by-row. Most citations in the sheet point at a chapter heading rather than a requirement—for two frameworks, essentially all of them—so a reader cannot look up the claim being made. Six hundred and ninety-two of the thousand rows carry a rationale repeated word-for-word on four or more rows, the mechanical signature of coding by block rather than by requirement. And every one of the 61 Exact ratings was challenged, which is consistent with this paper's own thesis: if no existing framework requires operator-independent evidence, the Exact column should be close to empty.

The pass cost a few minutes of compute. A cross-model audit with a stated acceptance rule is worth running for any coverage mapping, coding study, or requirements traceability matrix where the author and the coder are the same party—not because a second model is an authority, but because it is an independent generator of specific, checkable objections. It produces no κ, it does not replace human coders, and it should never be applied wholesale. The prompt, the output schema, the 192 proposals with per-item confidence, and the record of what was applied and what was refused are in the repository, so the pass can be repeated or contradicted.

8.3 What came out

Figure 4 and Table 3 have the results, and two patterns are worth staring at.

The first is that the Partial column is enormous and the Exact column is nearly empty. *Five of the eight frameworks have zero exact matches*, and the NIST risk framework—which scores 60%, second highest overall—is one of them. Read that again, because it is the whole story of this paper in one number. These frameworks ask for nearly every control we ask for. What they do not ask for, essentially anywhere, is machine-made evidence that the control actually held, checkable by someone who does not trust you.

The second is where the None column bunches up: in the chapters about evidence properties, grading, and disclosure. No framework we coded grades evidence by how independently it can be checked, or requires you to write down what you are still asking people to take on faith.

Framework	Exact	Partial	None	Coverage
OWASP AISVS [41]	16	63	48	62%
NIST AI RMF [35]	0	75	52	59%
ISO/IEC 42001 [37]	0	72	55	57%
SOC 2 [39]	0	68	59	54%
EU AI Act [40]	0	65	62	51%
CSA AARM [23]	13	47	67	47%
Zero Trust (SP 800-207) [17]	8	46	73	43%
MITRE ATLAS [42]	0	38	89	30%

Table 3: Coverage of all 127 requirements, coded individually by one coder. The thin Exact column is the finding: frameworks want the controls, not operator-independent evidence that the controls held.

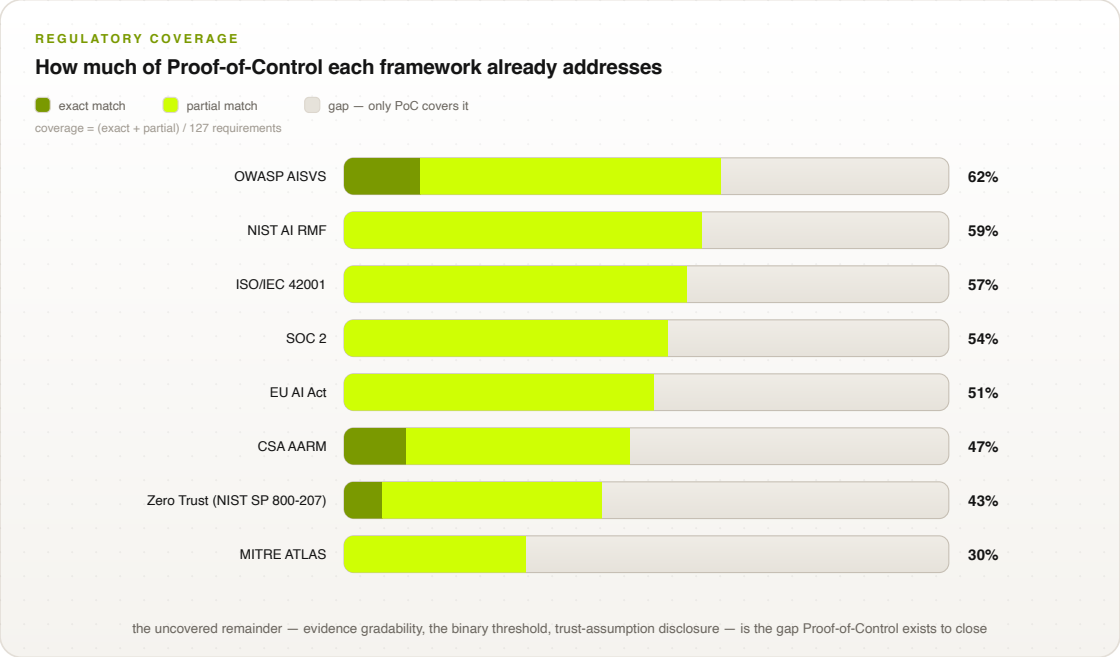

Figure 4: How much of Proof-of-Control eight frameworks already require. Dark: exact matches. Light: partial. The rest is the gap.

8.4 Why you should not lean too hard on these numbers

Now, a warning, and we mean this seriously. It is very easy to fool yourself with a table like that, and the person most likely to be fooled is us.

The measurement points the way we wanted it to. We asked how much of *our* list other people cover. Invent any sufficiently novel list and everyone else scores badly on it. So this table says we are working somewhere different. It does not say we are working somewhere *useful*. That is a separate argument and these numbers do not make it.

One person coded it. There is no measure of agreement at all. The published rubric specifies what a real study looks like: two independent coders per framework, Cohen's κ reported [59], at least one coder who did not write the requirements, and disagreements resolved by a third. That study is planned, and when it is done it replaces Table 3.

The cross-model audit in §8.2 is a partial substitute and not a replacement: it yields no measure of agreement, and its findings on citation quality and block-level coding apply to the sheet behind Table 3 as it stands.

The grain is coarser than it looks. Coding is per requirement, but most sections are still coded uniformly across the requirements they contain, so the apparent resolution of the sheet exceeds its real resolution. The published crosswalks now summarize each section over the requirements beneath it and name the ones a framework does not reach.

We only looked one way. We did not measure what those frameworks require that *we* miss—and there is plenty: organizational governance in the NIST framework, management-system obligations in ISO/IEC 42001, availability and processing integrity in SOC 2. We are deliberately narrower than all of them. An honest comparison shows both directions, and ours only shows one so far.

8.5 Compared with the closest things

Table 4 compares us against the systems we are most often confused with. The comparison is on *specified* properties—what each approach requires an implementation to do—because that is the only fair comparison available. ACS and AARM [21, 22, 23] specify interception and enforcement, then leave the record with the operator. Transparency logs [9, 30] have the non-equivocation machinery but know nothing about agents. Confidential-computing pipelines [20] attest computation but have no notion of an agent's authority or delegation. What we are contributing is the combination and the grading rule, not any one of the parts.

Property	**Prompts / guardrails**	**Static RBAC**	**ACS / AARM**	**Transparency logs**	**Proof-of-Control**
Every action goes through it	No	At the API layer	Yes	N/A	Yes
Decisions consider the path	No	No	Partly	N/A	Yes
Record integrity	None	Operator's logs	Operator's receipts	Chained, gossiped	Chained + anchored
A stranger can check it	No	No	With operator access	Yes	Yes (Tier 3+)
Everyone sees one history	No	No	Unspecified	Yes (with gossip)	Required at Tier 3+
Evidence describes what happened	No	No	Unspecified	N/A	Required (mediation)
Says what you still trust	No	No	No	Implicitly	Standardized

Table 4: What each approach *requires*. We are a specification; no claim is made here about deployed performance.

9 Does It Actually Work?

Talking about a design is easy. We built it and measured it, because that is the only way to find out whether the arguments above survive contact with a computer. The code is open source in the specification repository [3], at `https://github.com/AAI-Society/ov-poc-standard`.

What is real, and what is pretend. Real: the signing and verification, the hashing and chaining, the canonical descriptions, the path-aware policy over a bounded summary, the tickets and the far-end checks, the anchoring, the gossip, and the fail-closed behavior when the evidence store goes away. Pretend, *in this section*: the hardware. The measurements below use an enclave that is an in-process object whose key is never handed out and whose "measurement" is a digest of the policy code. That reproduces the structure the theorems are about, but not hardware isolation. **So the timings below leave out the cost of crossing into a real enclave, and the real number is bigger.** We would rather report a lower bound we measured than a total we invented—and we have since measured the difference rather than leaving it as a caveat: §9.10 runs the same pipeline inside a real Intel TDX trust domain and puts the trust-domain overhead at 6.2%. Everything below: Apple M2 Max, Python 3.11, one core, 5,000 iterations after 200 warm-ups.

9.1 The attacks

We wrote eleven attacks and ran each one twice: once against a system missing the relevant requirement, once against a system with it (Table 5). All eleven work when the requirement is missing and all eleven are refused or caught when it is there. The last three were not planned. A9 fell out of building the Merkle experiment in §9.11; A10 and A11 came from an independent review of the implementation, and both were live defects in it.

	Attack	**Without the requirement**	**With it**
A1	Snapshot substitution	Works first try: the record describes a harmless action while a malicious one runs	Refused: request does not match the evidenced description
A2	Editing the log	Works against unsigned operator logs	Caught: bad signature at record 2
A3	Dropping a step	Works: a dropped step looks like a step that never happened	Caught: gap at step 3
A4	Hiding recent history	Works: the shortened history is internally perfect	Caught: published total covers 8 steps, 5 shown
A5	Two sets of books	Works: each verifier's copy is internally perfect	Caught: different totals at the same index
A6	Read then send	Works: each action is inside the grant	Blocked: egress after a confidential read
A7	Reusing a ticket	Works without single-use tickets	Refused: nonce already used
A8	Evidence pipeline down	Works: the system keeps acting while logging is broken	Refused: fails closed, failure recorded
A9	Rewriting history, by someone who holds the key	Works: the rewritten history replays perfectly—every signature valid, every link correct, because the operator recomputed them all	Caught: the head presented does not match the total published *before* the rewrite
A10	Substitution past a check that is switched on but unconfigured	Works: the endpoint verifies signature, measurement, resource and nonce, and still runs a different request	Refused: the ticket carries a digest of the action the endpoint can recompute alone, and the check refuses when it cannot
A11	A valid ticket shown beside the wrong record	Works: both objects verify on their own, so the reconciliation balances	Caught: the ticket names the step and record it was issued with

Table 5: The attack harness. A1 is Proposition 1 in the flesh: without the far-end check, the attacker wins on the first attempt. A10 is the same proposition in the form it takes in practice—the check present, enabled, and doing nothing.

9.2 What it costs

Table 6 breaks a single intercepted action into pieces. The signing is 87% of it. The policy evaluation—the part everyone worries about—is 0.37 µs, which is nothing. The total, 201 µs, is 1.3% of our 15 ms budget. Put the other way: there is 74 times more room than we are using, and that room is what a real enclave transition would eat into. One core does 4,963 actions a second.

(The total is bigger than the sum of the parts because a full step signs twice —once for the record, once for the ticket—and serializes the token.)

Piece	Mean	p95	p99
Building and hashing the description	3.72 µs	4.21 µs	8.00 µs
Deciding (path-aware)	0.37 µs	0.46 µs	1.12 µs
Signing (Ed25519)	84.25 µs	97.96 µs	110.12 µs
Adding the link, and the tree root	0.73 µs	0.92 µs	3.71 µs
One whole action	201.47 µs	230.58 µs	254.75 µs

Table 6: Cost of one intercepted action, not counting enclave transitions.

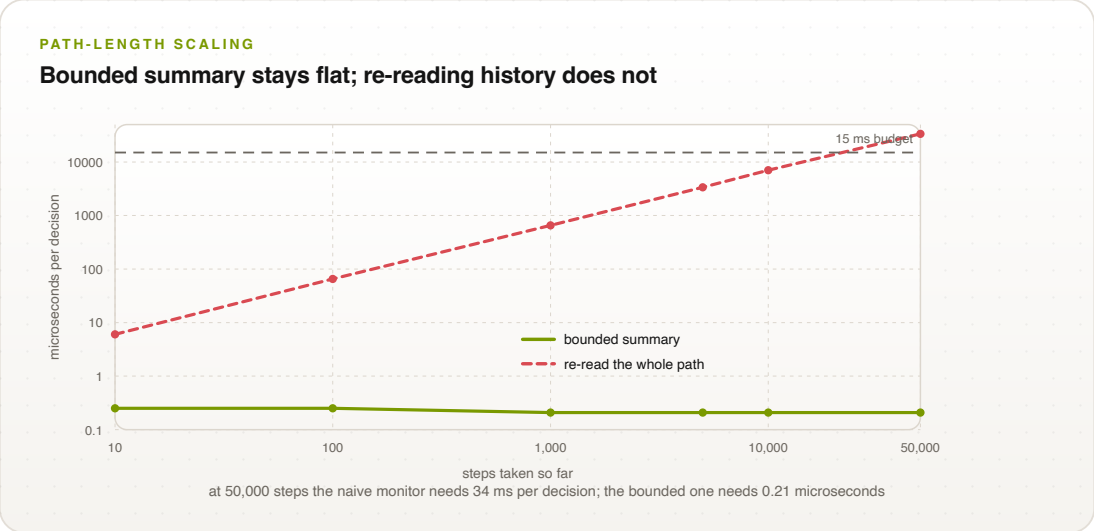

Figure 5: Deciding one action, at depth, on log axes. The bounded summary is a flat line across four orders of magnitude; re-reading the whole path is a straight diagonal that crosses the 15 ms budget at roughly 22,000 steps.

9.3 Does it stay cheap as the agent runs longer?

Section 6.2 claimed the bounded summary makes the per-step cost independent of how long the agent has been going. That is the kind of claim that is easy to make and easy to be wrong about, so we measured it (Table 7). From 10 steps to 50,000—a factor of five thousand—the cost per step does not move: it sits at 0.21 µs throughout (Figure 5). The naive version, which re-reads the whole history every time, grows exactly as you would expect and would blow the entire 15 ms budget somewhere around 22,000 steps.

So this is not bookkeeping. It is the difference between a monitor that survives a long-running agent and one that gets slower until somebody turns it off.

Steps so far	Bounded summary	Re-reading everything
10	0.208 µs	5.79 µs
100	0.208 µs	64.92 µs
1,000	0.208 µs	663.88 µs
5,000	0.209 µs	3379.17 µs
50,000	0.208 µs	34 368.50 µs

Table 7: Cost of deciding one action, at depth. Flat versus linear.

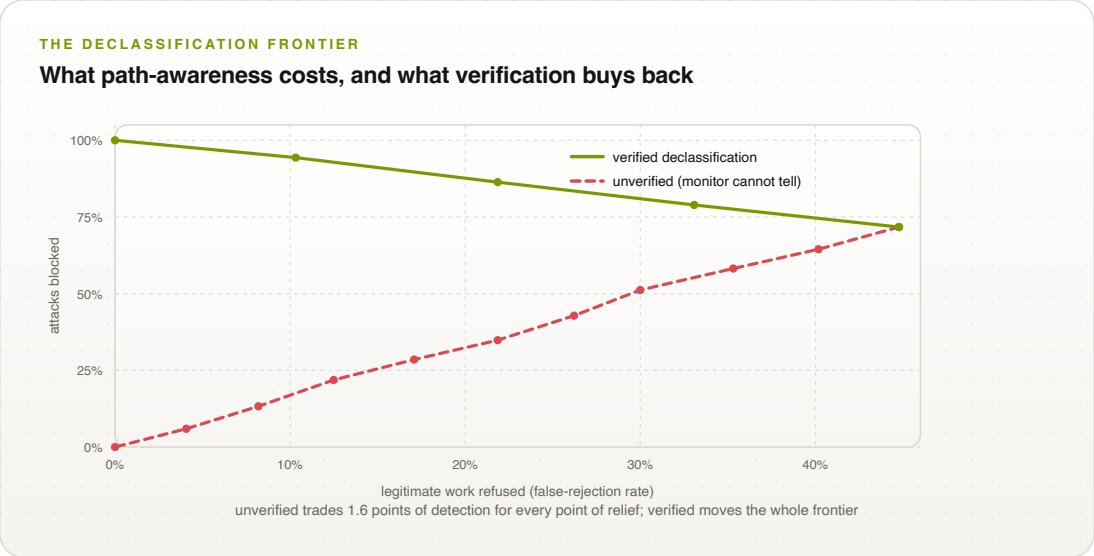

Figure 6: The declassification frontier. Dashed: declassification the monitor cannot verify—every point of relief from false rejections costs about 1.6 points of detection, and there is no knee to aim for. Solid: declassification the monitor can verify, where both quantities improve together.

Checking, from outside, costs 190 µs per record and grows linearly: a stranger can verify a 5,000-step history in under a second using nothing but published keys and reference values. The important word there is *stranger*. No cooperation from the operator is required at any point.

9.4 The measurement we did not expect

Everything above says the machinery is cheap. This section says the policy is expensive, in a way no amount of faster code will fix.

We ran 2,000 randomized workflows—70% ordinary work, 30% trying the read-then-send trick—past the path-aware monitor and counted how often it refused work it should have allowed. With no way to clear an accumulated label, it refused **42%** of perfectly legitimate work. Not a bug: we told it "don't send after reading something sensitive," and it enforced exactly that, forever, because nothing ever cleared the flag.

The standard remedy, from fifty years of information-flow work [53], is a *declassification point*: a place where data is redacted and the accumulated label is reset. Adding one took the false rejections to zero.

9.5 Where we fooled ourselves, and the fix

That result was too good, and it was too good because we cheated without noticing. In that first experiment only the *benign* workflows were allowed to declassify. A real attacker would of course use any declassification point available to it—that is precisely what makes declassification hard, and we had quietly assumed the problem away.

So we ran it again properly. This time a declassification point is available to whoever reaches it, and the monitor—which sees only that a redaction step occurred, and cannot tell a genuine redaction from a sham—treats it the same either way. We swept the *coverage*: what fraction of workflows have a declassification point before egress. Figure 6 has the result, and it is much less flattering.

The dashed line is a straight trade, not a bargain. Going from no declassification to full coverage takes false rejections from 44.8% to zero and takes detection from 71.7% to zero along with it—**1.6 points of detection lost for every point of relief**. There is no knee. There is no good place to sit. An unverified declassification point is not a solution to label creep; it is a dial between two different ways of failing.

The fix is not to tune the dial. It is to make declassification something the monitor can check: only a trusted redaction tool clears the label, and it really removes the content, so an attacker routing through it gains nothing because the data it wanted is gone. That is the solid line, and it does what you want in both directions at once: at full coverage, false rejections reach zero *and* detection reaches 100%, because a redaction that genuinely redacts defeats the exfiltration it was supposed to launder.

This changes a requirement, not a setting. **A deployment that mandates path-aware authorization must specify not only where declassification happens but how the monitor verifies it.** Declassification by annotation—a workflow asserting "this is fine now"—measurably destroys the detection the whole mechanism exists to provide.

We also checked whether the size of the bounded summary mattered, sweeping the label bound B from 1 to 16. It made no difference at all on these workloads: the binding constraint is the declassification policy, not the size of the state we carry. That is a null result and we report it as one.

9.6 Reasons to doubt these numbers

The enclave is pretend here, so a real deployment costs more than 201 µs—by 6.2% for the trust domain itself, which §9.10 measures on real Intel TDX, plus 39.5 ms for each hardware quote, which is the cost that actually matters and is not in any number in this section. Everything runs in one process, so no network or IPC costs are included. The workflows are synthetic: the shape of the utility result is solid—label creep is real and declassification fixes it—but 42.2% is a property of our workload model and yours will differ. It is Python, so the absolute numbers are conservative; the breakdown between components is the part that will hold up. And we did not implement the privacy-preserving evidence options at all.

Four more, specific to the newer work. Everything is measured on *one* core with *one* log, and a hash chain is inherently serial—each link depends on the one before—so a single global log will cap throughput no matter how many cores you have. Per-agent logs parallelize but make cross-agent ordering, and therefore delegation, harder to reason about; the standard does not yet say which you must build, and it should. Our tree keeps every leaf in memory so it can generate proofs, which is fine for a log that stores the records anyway but is not a storage design. The CBOR claim keys are a provisional private-use allocation pending IANA registration, so the round-trip property is what we are asserting, not the integers. And our reference serializer is not quite RFC 8785: it orders keys by code point rather than by UTF-16 code unit, which is identical for the claim set, whose keys are all ASCII, and can differ for an application payload with a key outside the Basic Multilingual Plane. We publish a test vector that exhibits exactly that difference rather than describing it and hoping.

9.7 Signatures that have to outlive the attacker

There is one more thing worth measuring, and it comes from taking the purpose of the system seriously. Evidence is not a session key. A record made today may be looked at in a dispute, an audit, or a courtroom ten or twenty years from now, and it is worth exactly what its signature is

worth *at the moment somebody looks*. So a signature scheme with an expiry date does not merely expire. It reaches back and destroys the value of evidence about things that happened years earlier. For most systems post-quantum migration is about protecting future traffic; for an evidence system it is about protecting the past.

The good news is that most of the machinery does not care. The hash chain, the sequence numbers, and the anchoring are already post-quantum: the best known quantum attack on preimages is Grover's, which buys a square root, leaving SHA-256 at about 128 bits. So the structure survives. What has to migrate is the signatures—on tokens, on capabilities, and on published roots.

We measured what that swap costs (Table 8), using NIST's ML-DSA (FIPS 204) against Ed25519.

Scheme	Signature	Public key	Record	Per million actions
Ed25519 (classical)	64 B	32 B	1,336 B	1.34 GB
ML-DSA-44	2,420 B	1,312 B	10,760 B	10.76 GB
ML-DSA-65	3,309 B	1,952 B	14,316 B	14.32 GB
Hybrid Ed25519 + ML-DSA-44	2,484 B	1,344 B	11,016 B	11.02 GB

Table 8: Cost of post-quantum evidence signatures. Records carry two signatures (token and capability), hex-encoded. Sizes are implementation-independent; see the caveat below about timing.

The headline is size, not speed. A signature goes from 64 bytes to 2,420—a factor of 38—and because an evidence chain stores one per action, the whole record grows about eightfold, from roughly 1.3 GB to 10.8 GB per million actions. Before you accept that bill, note that switching from the JSON rendering to the CBOR one halves every figure in the table (§5.6), because a signature stored as hexadecimal text costs twice what the same signature costs as bytes. The encoding choice is worth more than the algorithm choice costs. That is the real bill, and it is a storage and bandwidth conversation, not a cryptography one.

A caveat we want to be loud about. Our Ed25519 is a C-backed library and our ML-DSA is a pure-Python reference implementation, so the timings are not comparable and we are deliberately not putting them in the table. Pure-Python ML-DSA signing measured tens of milliseconds, which would blow the whole latency budget—but that number says something about our Python, not about ML-DSA. Optimized implementations are competitive with Ed25519, and verification in particular is fast. Anyone who cites a timing comparison from this section is citing an artifact. The size numbers are the ones that transfer.

Figure 7 puts that in units anyone can budget for. A fleet of a thousand agents taking a hundred actions each per day, kept for the seven years a regulator might ask about, comes to 0.24 TB classically and 2.65 TB post-quantum. The multiplier is 38 on the signature and about eleven on the whole record; the absolute number is a couple of terabytes, which is to say the frightening-sounding factor turns out to be a rounding error on a storage invoice. We would rather have measured that than argued about it.

What we changed in the standard. Two requirements. First, every record and capability must name the algorithm that signed it, and a claim must declare a migration path—so a verifier knows what to check and an operator can change schemes without orphaning evidence already published. Second, if the declared retention period runs past the period the signature scheme is expected to survive, then the implementation either signs post-quantum or hybrid, or re-signs and re-anchors

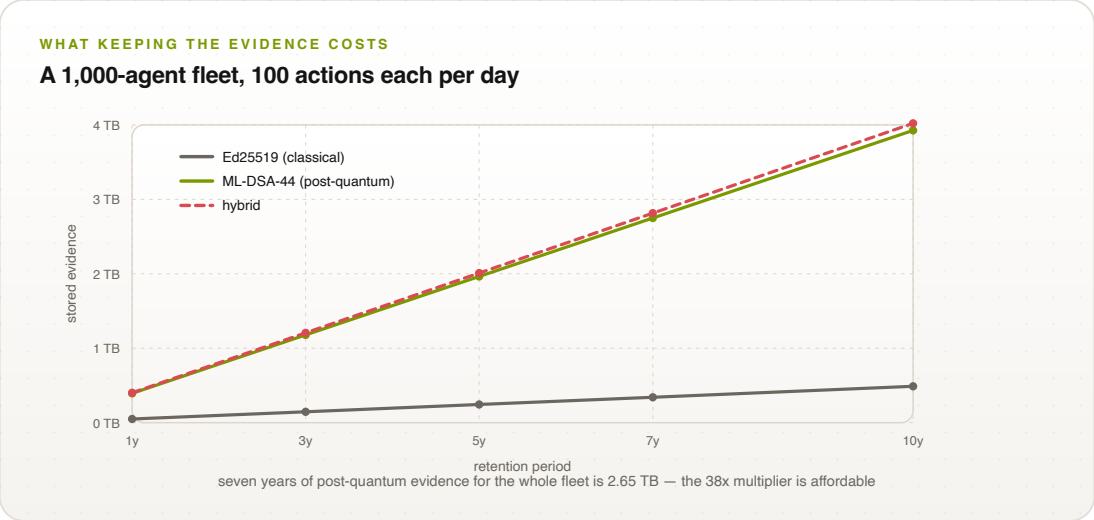

Figure 7: What keeping the evidence costs, for a realistic fleet. Post-quantum signatures multiply the bill by about eleven; the bill is small enough that it does not matter.

retained evidence before the projection lapses. Hybrid is the sensible default during transition: you pay 2,484 bytes instead of 2,420 and you are covered whichever way the next decade goes.

9.8 Choosing the anchoring interval, by measurement

Earlier we said the publishing interval Δ bounds how much recent history an operator can hide, and we offered a table of suggestions. Suggestions are unsatisfying in a paper that measures everything else, so we measured this too.

Publishing one root costs 85.7 µs—one signature and an append. Our gateway sustains about 4,989 actions per second on one core. Those two numbers determine everything (Figure 8): the blind window is the action rate times Δ, and the overhead is the publication cost divided by Δ.

The numbers are friendlier than we expected. Publishing every second leaves at most about 5,000 actions inside the blind window and costs 8.6×10^{-5} of wall-clock time—under a hundredth of a percent. Publishing every millisecond shrinks the window to five actions and still costs under 9% of one core. Publishing hourly is essentially free and leaves 18 million actions hideable, which for anything irreversible is absurd.

The guidance becomes arithmetic rather than taste: *decide how many actions you could tolerate an adversary concealing, divide by your action rate, and that is your* Δ. For irreversible actions the tolerable number is near zero, which is why we recommend publishing per action there and paying the round trip.

9.9 Getting rid of the signing cost

Signing is 87% of a step, which invites an obvious question: why sign every action instead of one root over a batch of them? We measured that too (Figure 9).

Batching 128 actions under one signature makes each action **64 times cheaper**—from 85.7 µs to 1.3 µs, and from 11,663 to 745,573 actions per second on one core. The price is that an action is not covered by a signature until its batch closes, which at that size is at most 170 µs.

That is the same shape of tradeoff as Δ, one level down: a small window in which the record exists but is not yet cryptographically fixed. If the machine dies inside that window those records

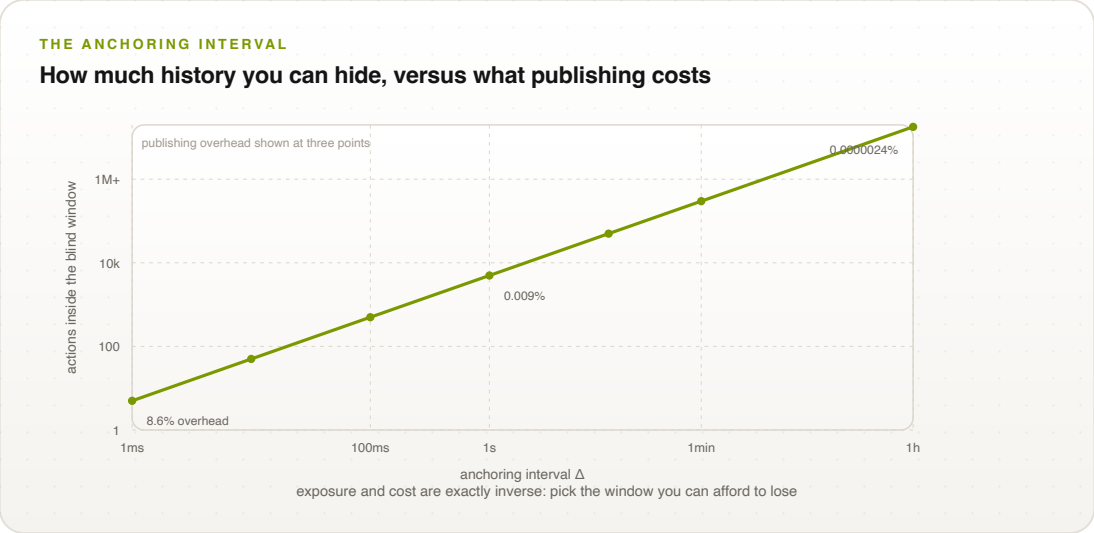

Figure 8: The anchoring tradeoff, measured. Exposure and cost are exactly inverse, so the only question is which one you would rather spend.

are lost, and for evidence that is the same as their never having existed. So batch size is not a performance knob to be turned as far as it goes—it is a second exposure window, and it belongs next to Δ in the conformance claim. Small for irreversible actions, generous for internal steps, on exactly the same reasoning.

The useful conclusion is that the dominant cost in our measurements is not inherent. Anyone who reads Table 6, concludes "signing is too expensive," and starts redesigning should read Figure 9 first.

9.10 On real hardware, at last

Everything above was measured with the enclave as an in-process object, and we have been careful to say so. That stand-in reproduces the protocol but leaves the load-bearing claim untested: that a verifier can establish *which code produced this evidence* without trusting the operator. So we ran the pipeline inside a real Intel TDX trust domain on Google Cloud, with an identical non-confidential instance of the same shape as a control, so that the cost of the trust domain is isolated rather than confounded with a change of CPU.

The binding. A quote on its own proves that some measured environment exists. It says nothing about which signing key belongs to it, and an operator who could pair an honest quote with a key held outside the domain would defeat the whole arrangement. TDX provides 64 bytes of `REPORTDATA` that the hardware copies into what it signs, so we put the digest of the evidence public key there. The hardware signature then covers the measurement *and* the key together, and the statement a verifier receives is the one that matters—this key lives inside this measured environment—rather than two facts that merely arrived in the same envelope.

It works. The instance reports `Intel TDX` and `tdx: Guest detected`; a quote is 8,000 bytes and carries a real MRTD (`c1ee9c16e3af…`); the quote commits to our key, and rejects a key we did not attest. This is requirement C7.2.4, and it is the first claim in this paper established by hardware rather than by argument.

Figure 9: Batch signing. Throughput rises almost linearly with batch size while the window in which an action is not yet covered by a signature stays in the microseconds.

The cost, which is not where we expected it. Table 9 has the numbers, and they separate cleanly into two very different quantities.

	Measured	Relative
Software pipeline, TDX off (control)	152.88 µs	—
Software pipeline, inside the TD	162.33 µs	+6.2%
Trust-domain overhead	9.45 µs	**6.2%**
One hardware quote	39.5 ms	**4,180× the overhead**

Table 9: The pipeline on Intel TDX (GCP `c3-standard-4`) against an identical non-confidential instance. Running inside the trust domain is nearly free; attesting it is not.

Running *inside* a trust domain costs 6.2%—about ten microseconds a step. That is the number we had no way to measure before and were careful not to guess at, and it is small enough to be uninteresting, which is the best kind of result for a design that depends on it.

Producing a *quote* costs 39.5 ms. That is about four thousand times the per-step overhead, and 2.6 times the entire 15 ms budget for a single action. Attesting once per action is therefore not expensive but impossible, and no amount of engineering will close a gap of that shape. Every real deployment will take one quote and use it for many actions: amortized over 100 actions the cost falls to 557 µs a step, and over 1,000 to 202 µs, which is back inside the budget.

Which creates an exposure window we had not named. If a quote is taken once and covers the next thousand actions, then those actions rest on a measurement made before them. Should the measured code change in between, their evidence attests an environment that is no longer the one that ran, and nothing in the record says so.

This is the same shape as the anchoring interval Δ of §9.8: a parameter that trades cost against how much can happen unobserved, where the only wrong answer is not to state it. So it becomes a requirement in the same form. C7.2.3 obliges a conformance claim to declare a maximum attestation refresh interval and to refresh within it or stop. We would not have thought to write

that requirement from the software model, because in the software model a measurement is free and staleness never arises.

What this still does not establish. The reviewers of this paper asked for a real TEE, and this is one, but it answers a narrower question than the one they were asking. It shows the protocol runs on real hardware, what the hardware costs, and that the key binding holds. It does *not* demonstrate resistance to a hostile host: we did not compromise a hypervisor, and Google operates the one underneath this. The adversary of §7 owns the OS; what we have shown is that the construction works on hardware that claims to resist such an adversary, not that the resistance holds.

And who you must still trust. The most useful output of running on real hardware was the list it forced us to write down. To believe a measurement from this deployment, a verifier accepts: Intel, that the TDX module and CPU behave as specified and the provisioning key is not misissued; Intel's certification service, that the collateral fetched to check the quote is authentic; the quoting enclave, that it signs reports honestly; Google, that the host does not extract domain memory by some route outside the TDX threat model, and that the firmware measured into the runtime registers is what it says; and whoever publishes reference values, that the MRTD being compared against belongs to the code one believes it does.

Anchoring the evidence log removes *none* of these. It makes the *history* publicly auditable, which is a real and separate property. The *measurement* is still established by that chain of parties. Our tier definitions said that at Tier 3 there is nobody left to trust, and this list is the plain refutation of that sentence. The standard has since been corrected against it—the Tier 3 column no longer claims the trusted party is removed—and we take up what that changes, and what it does not, in §10.

9.11 The question a chain cannot answer

Everything so far treats verification as an all-or-nothing act: you take the history and you replay it. That is fine when the history is a few thousand records. It stops being fine the moment you ask the question a real auditor asks, which is not "is the whole log intact?" but "was *this* action in the log?"

With a hash chain there is no way to answer the second question except by answering the first. Each link depends on the one before, so to convince yourself that record 500,000 is genuine you fetch all million records and rehash them. An auditor spot-checking one action has to download the entire log. That is not an inefficiency you optimize away later; it decides whether audit is something you do continuously or something you do once a year with a data-transfer budget and a sense of dread.

The fix is not ours and it is not new. Certificate Transparency solved this in 2013 with an append-only Merkle tree [9], and we have been citing CT throughout this paper while quietly using a structure that gives up its main advantage. So we built the tree and measured what it buys.

Checking one record. Table 10 is the whole argument. An inclusion proof at 100,000 records is 544 bytes and takes 7 µs to check. Replaying the chain to learn the same fact takes 70 ms and 123 MB of transfer—about 69,000 times more data (Figure 10). And the gap is not fixed: one curve is linear and the other is logarithmic, so every record you add makes the comparison worse.

Put it in operational terms. A thousand agents at a hundred actions a day produce 36.5 million records a year, a tree 26 levels deep. Checking one sampled action costs about 2 kilobytes.

Records	Replay	Check a proof	Speedup	Proof	Fetched to replay
10	8 µs	1.87 µs	4×	128 B	12 kB
1,000	672 µs	4.20 µs	160×	320 B	1.2 MB
10,000	6.9 ms	5.67 µs	1,217×	448 B	12.3 MB
100,000	70 ms	7.01 µs	**9,972**×	544 B	122.7 MB

Table 10: Establishing that one named record is in the log. The chain must be replayed from the beginning; the tree needs a proof of $\lceil \log_2 n \rceil$ hashes.

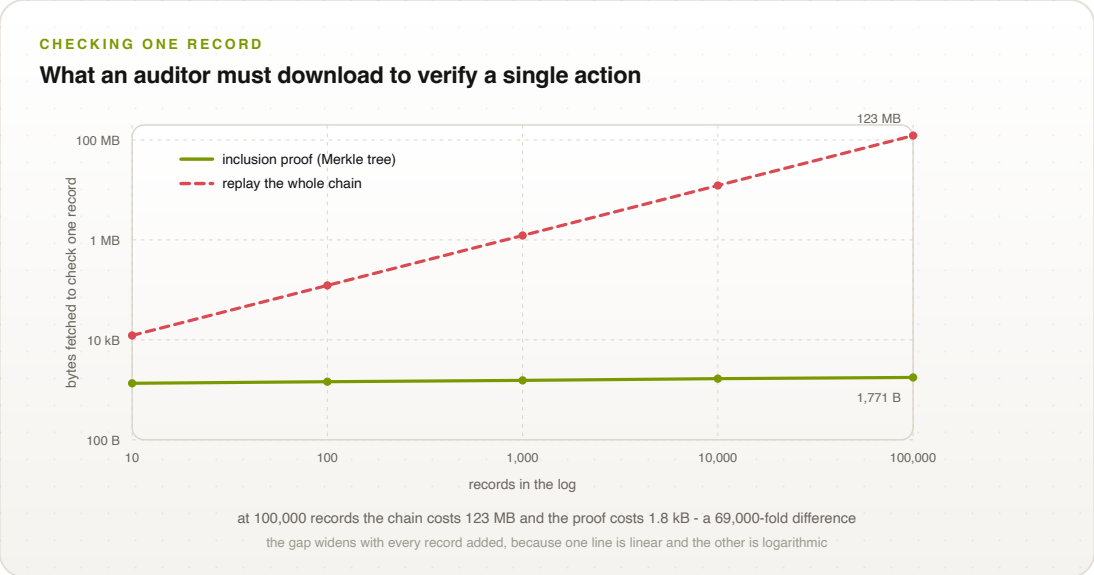

Figure 10: What an auditor must download to check a single action. One line is linear in log size and the other is logarithmic, which is why the gap widens with every record written.

Replaying the chain costs 44.8 GB—and costs the same whether you wanted to check one record or a million, because replay is all-or-nothing. Proofs cost what you actually asked for.

What it costs the agent. A structure that helps the auditor at the agent's expense would be a bad bargain, so this is the number that decides adoption. Maintaining the tree adds 2.1 µs per action at ten records and 3.2 µs at a hundred thousand—between 2.4% and 3.8% of the 85 µs a signature already costs. It is close to free, and it stays close to free as the log grows, because appending touches $\log_2 n$ nodes rather than n.

The part where it broke our own system. We expected this experiment to produce a performance table. It produced attack A9 instead, and A9 is the most useful thing in this section.

Every attack in Table 5 except this one assumes the adversary cannot sign. But the operator running the enclave *can*—that is what operator-controlled infrastructure means, and it is the threat model we claim to be working in. Give an operator the key and it can reach into the middle of the log, change a record, recompute every link after it, and re-sign the whole thing. What comes out has exactly the right length. Every signature verifies. Every link matches. Replaying it finds nothing wrong, because there *is* nothing internally wrong with it: it is a perfectly consistent history of events that did not happen.

The only thing in the world that contradicts it is a root published before the rewrite. Our verifier had one—and compared the published *step count* while never comparing the published *root*. So it accepted the forgery, cheerfully, and told us the chain verified.

We had written the truncation check and stopped thinking. Truncation is the attack where the history is too short, so we checked the length, and checking the length felt like checking the anchor. It is not. Comparing roots catches the rewrite immediately, and a consistency proof does better still: it settles in $O(\log n)$ not merely *that* two published roots disagree but that the newer history is not an extension of the older one—which is the precise statement that history was rewritten. Those proofs run 224 to 480 bytes and verify in under 9 µs, and in our harness they catch every rewrite we could construct.

This is now requirement C7.3.5, and its wording—"compares the recomputed root against the anchored value rather than only against a step count"—is oddly specific for a standard because it is a description of our own mistake. We would rather the requirement be strange and correct than elegant and silent about the thing that actually goes wrong.

There is a general lesson here that we did not expect to be writing down. Two earlier findings in this paper came from proving theorems; this one came from building a component we thought was an optimization. Implementing the fast path forced us to state precisely what a published root was supposed to guarantee, and stating it precisely was enough to notice that we were not checking it. The performance work found a security bug, which is not the direction that traffic usually runs.

9.12 Remaining parameters

For the **privacy path**, an implementation admitting zero-knowledge evidence must state the relation proven, the setup assumption, and what the disclosed statement leaks. "We use ZKPs" is not a claim anyone can check—and if the setup was run by one party, you have quietly reintroduced exactly the trusted party the whole scheme exists to remove.

10 What We Still Don't Know

Standards-integration targets and the phased rollout are in Appendices A and B. This section is for the things we have not settled.

10.1 The honest list

(1) The enclave is no longer pretend, and the honest limit has moved. An earlier draft of this list said the hardware was not there and that measuring on TDX or SEV-SNP was the obvious next job. It has since been done: §9.10 reports the pipeline running inside a real Intel TDX trust domain on Google Cloud, against an identical non-confidential instance as a control. Running inside the trust domain costs 9.45 µs a step, 6.2%—the number we had refused to guess at, and small enough to be uninteresting. Producing a quote costs 39.5 ms, which is 2.6 times the entire per-action budget and made per-action attestation not expensive but impossible; that is where requirement C7.2.3 came from, and we would not have thought to write it from the software model, where a measurement is free and staleness never arises.

What remains unmeasured is narrower and should not be read as settled. We did not compromise a hypervisor, and Google operates the one underneath this deployment, so what we show is that the construction works on hardware that claims to resist a hostile host—not that the resistance holds. The adversary of §7 owns the OS, and that adversary has not been run against this.

SEV-SNP and ARM CCA are unmeasured entirely, and a second platform is the obvious next job now.

Our tier definitions overclaimed, and we now have the receipts. The standard said that at Tier 3 there is nobody left to trust. §9.10 lists five parties a verifier must still accept to believe a measurement from our own TDX deployment, and anchoring removes none of them. The sentence conflates two properties that a careful reader will separate for us: evidence can be *publicly verifiable*—anyone may check it with published tools—without being *trust-independent*, and hardware attestation buys the first while leaving a vendor chain intact underneath the second.

That sentence is now gone. The Tier 3 column says the evidence is produced by the mechanism and checkable by anyone with published tools, and that the parties which remain load-bearing are disclosed rather than removed; the word "trustless" has been struck from the normative text, where it occurred once. What has *not* been settled is the framing that replaces it—whether the Tier is demoted from the primary claim to a coarse summary, with what must be trusted carried by the trust-assumption disclosure instead, and whether the Tier-2/Tier-3 test is restated as who selects the trusted party and whether their dishonesty is publicly detectable. Those are working-group decisions and are marked as such.

We note what the correction rests on, because it bounds it: five self-authored deployment descriptions and no real-world system. That is enough to refute a universal claim—one counterexample does it, and there are two disjoint ones—and it is not enough to install a new ordering or a new taxonomy in its place. We would rather print that here than let it be discovered.

(2) We verify what happened, not whether it was wise. No claim about correctness, fairness, or intent. An agent that does something harmful entirely within its permissions is a safety problem, and we do not solve safety problems.

(3) Trust is reduced, not abolished. Everything above Tier 2 rests on (A1), (B1), (C1). Enclaves have been broken [33, 34], and the chip vendor is a party you are trusting. Our answer is to make you write that down, not to pretend it is not there.

(4) The framework comparison is one coder and one direction (Section 8.4). Treat it as a sketch.

(5) The hard line may move. Where exactly Tier 2 ends and Tier 3 begins is under review by people who do cryptography and complexity theory for a living, including work on what is provably impossible to verify efficiently. If they tell us the line is in the wrong place, the line moves.

(6) Failing closed means failing. At Tier 4, if evidence cannot be produced, work stops. That is the property we want and also an availability risk, and both facts have to be disclosed. You do not get to advertise only the first one.

Open questions the working group is still arguing about—who owns identity binding, whether unlinkable-but-verifiable identity should be allowed, whether evidence continuity across jurisdictions is a fifth property, and how continuous "continuous" has to be—are tracked publicly.

10.2 Where this stands

Proof-of-Control is a working draft under public comment, with a public repository (`https://github.com/AAI-Society/ov-poc-standard`) containing the specification, the audit checklist, the coding sheets, and the implementation with its attack harness and benchmarks. We would rather have this attacked than admired. The most useful thing anyone could do is run the utility measurement on real agent traces instead of our synthetic ones, because that number—not the cryptography—is what will decide whether this is deployable.

11 Conclusion

The way we have always known what software did is that we asked it and it told us. That worked when software followed instructions, and it does not work now that software makes decisions, at machine speed, in situations nobody enumerated in advance. Asking an agent what it did is not a check on the agent; it is a conversation with the party you would be checking.

What we have described is the alternative: evidence made by the mechanism while it acts, chained so gaps show, published so nobody can keep two sets of books, tied to the effect so it cannot describe a fiction, and gradable so a buyer can ask one yes-or-no question. We proved the parts that are provable, and we were careful to say which parts are not: an attester by itself proves less than it appears to, and a monitor that watches the path will refuse half your work unless you tell it where to forget.

We also built it, which is how we learned that the cryptography costs 201 µs and is not the problem, that the policy design costs 42% of your legitimate work and is, and that making the whole thing survive quantum computers is a storage bill rather than a redesign. Then we ran it on real Intel TDX hardware, which settled the question the software model could not: running inside a trust domain costs 6.2%, about ten microseconds a step, and is not where the expense lives. Producing one hardware quote costs 39.5 ms—2.6 times the entire budget for a single action—so attesting per action is not expensive but impossible, and every deployment will amortize one quote across many. That is an exposure window rather than an optimization, and naming it became a requirement we had no reason to write while the enclave was a stand-in. Building it is also how we found the bug in our own verifier, which no amount of rereading the specification had turned up: we compared a published step count where we should have compared a published root, and a rewritten history sailed straight through. We would not have noticed it by thinking harder. We noticed it by writing the code that made us say precisely what a published root was for.

The specification, the code, the attacks, the schema, the test vectors, and the numbers are all open. If any of it is wrong, it should be possible for you to find out—which, in the end, is the entire point.

Acknowledgments

This work is developed within the Proof-of-Control Initiative of the Advanced AI Society. We thank the domain working groups and contributors whose input shaped the draft standard, including Bob Blessing-Hartley, Advait Patel, David Thomson, Drummond Reed, and Hart Montgomery (Linux Foundation Decentralized Trust), and the Cloud Security Alliance MAESTRO and AARM communities. The Proof-of-Control Lab is being established as a community lab at Linux Foundation Decentralized Trust.

A Standards Integration

ISO/IEC JTC 1/SC 42: Proof-of-Control provides a technical realization for active SC 42 workstreams [60, 61, 62]—a technical annex for auditable runtime controls under ISO/IEC 42001 Clauses 8–9 (supplying cryptographically verifiable compliance evidence during certification audits) [37], operationalization of ISO/IEC 23894 risk treatment as executable, hardware-verified policy bundles [38], and an SC 42/SC 27 bridge coupling AI oversight with confidential-computing standards [63]. *IETF RATS:* an Internet-Draft establishing an AI-agent EAT profile [6, 25]—standardizing claims for lifecycle hook types, policy-bundle hashes, Agent Bill of Materials (Ag-

BOM) digests, and step indices—and an extension of multi-verifier attestation to sub-agent delegation chains [26]. *ACS:* Proof-of-Control supplies the missing cryptographic layer for ACS [21, 22], anchoring its decision engine in TEEs and signing decision tokens. *NIST AI 600-1:* Proof-of-Control provides concrete implementations for the generative-AI profile's measurement controls [36, 64, 65]—enforcing that deployment configurations cannot bypass security controls and satisfying continuous security verification with attestation tokens for every tool call and state transition.

B Implementation Roadmap

Figure 11 summarizes the path to ratification and adoption. **Phase 1 (months 1–6): open specification and reference core**—core schema extensions on the ACS repository, open-source Rust reference enclave policy engines, and the IETF Internet-Draft for the AI-agent EAT profile. **Phase 2 (months 7–12): multi-vendor interoperability and SDO balloting**—SC 42 working groups, conformance testing across Python, Node.js, and Rust agent frameworks, and telemetry integration with OpenTelemetry and OCSF. **Phase 3 (months 13–24): ratification and deployment**—normative annexes in ISO/IEC 42001 and 23894, RFC publication, and deployments across financial, healthcare, and public-sector workloads. On the adopter side, the same three phases carry an organization from foundational key, signing, and logging infrastructure through expanded proof coverage to continuously monitored operation—each phase reaching readiness for one conformance stage.

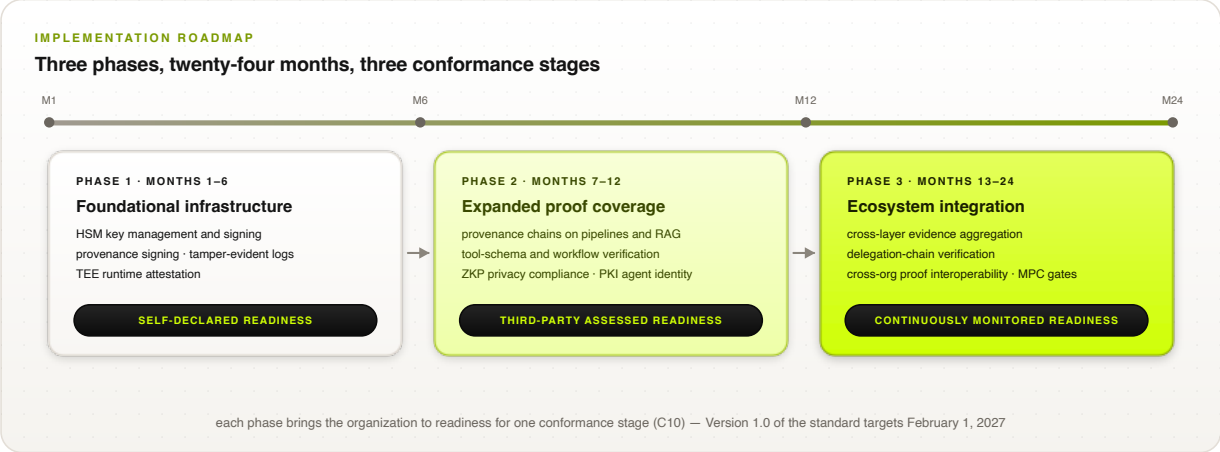

Figure 11: Implementation roadmap. Each phase brings an adopting organization to readiness for one conformance stage; on the standards track, Version 1.0 targets February 1, 2027.

References

[1] Palo Alto Networks. A complete guide to agentic AI governance. `https://www.paloaltonetworks.com/cyberpedia/what-is-agentic-ai-governance`, 2026.

[2] Airia. What is AI agent governance? A framework for keeping agents in check. `https://airia.com/blog/what-is-ai-agent-governance-a-framework-for-keeping-agents-in-check/`, 2026.

[3] Advanced AI Society. Open verification: the proof-of-control standard for agents, working draft v0.1.4. `https://github.com/AAI-Society/ov-poc-standard`, 2026. Specification repository: requirement chapters C1–C10, appendices, audit checklist, and regulatory coding sheets. Society homepage: `https://advancedaisociety.org/`.

[4] Christian Catalini, Xiang Hui, and Jane Wu. Some simple economics of AGI. *arXiv preprint arXiv:2602.20946*, 2026. `https://arxiv.org/abs/2602.20946`.

[5] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, Internet Engineering Task Force, 2023.

[6] Laurence Lundblade, Giridhar Mandyam, Jeremy O'Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, Internet Engineering Task Force, 2025.

[7] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO '87*, pages 369–378. Springer, 1988.

[8] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium*, 2009.

[9] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013.

[10] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[11] Joseph A. Goguen and José Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy*, 1982.

[12] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[13] Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. AgentHarm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[15] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

[16] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024.

[17] Scott Rose, Oliver Borchert, Stu Mitchell, and Sean Connelly. Zero trust architecture. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020.

[18] NETBankAudit. Generative AI controls and risk assessments for financial institutions. `https://www.netbankaudit.com/resources/generative-ai-controls-risk-assessments`, 2026.

[19] Red Hat. Attestation in confidential computing. `https://www.redhat.com/en/blog/attestation-confidential-computing`, 2026.

[20] EQTY Lab. Verifiable compute: AI integrity suite. `https://www.eqtylab.io/blog/verifiable-compute-press-release`, 2026.

[21] Agent Control Standard. Agent control standard launches open framework for runtime governance of AI agents. `https://www.businesswire.com/news/home/20260527326259/en/Agent-Control-Standard-Launches-Open-Framework-for-Runtime-Governance-of-AI-Agents`, 2026.

[22] Microsoft Command Line. Agent control specification: Portable runtime governance for AI agents. `https://commandline.microsoft.com/agent-control-specification-runtime-governance/`, 2026.

[23] Cloud Security Alliance. Autonomous action runtime management (AARM). `https://cloudsecurityalliance.org/`, 2026. Contributed by Vanta.

[24] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote attestation procedures architecture. Internet-Draft draft-ietf-rats-architecture-07, Internet Engineering Task Force, 2021.

[25] Ned Smith. TCG attestation framework and IETF remote attestation procedures. `https://trustedcomputinggroup.org/wp-content/uploads/2_N.Smith_TCG-JRF02292024.pdf`, 2024. Trusted Computing Group.

[26] IETF RATS Working Group. Remote attestation with multiple verifiers. Internet-Draft draft-ietf-rats-multi-verifier-00, Internet Engineering Task Force, 2026.

[27] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, Internet Engineering Task Force, 2021.

[28] Linus Nordberg, Daniel Kahn Gillmor, and Tom Ritter. Gossiping in CT. Internet-Draft draft-ietf-trans-gossip-05, Internet Engineering Task Force, 2018.

[29] Melissa Chase and Sarah Meiklejohn. Transparency overlays and applications. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.

[30] Kirill Nikitin, Eleftherios Kokoris Køgias, Philipp Jovanovic, Nicolas Gailly, Linus Gasser, Ismail Khoffi, Justin Cappos, and Bryan Ford. CHAINIAC: Proactive software-update transparency via collectively signed skipchains and verified builds. In *26th USENIX Security Symposium*, 2017.

[31] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, ESD/AFSC, Hanscom AFB, 1972.

[32] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[33] Victor Costan and Srinivas Devadas. Intel SGX explained. Cryptology ePrint Archive, Report 2016/086, 2016.

[34] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F. Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution. In *27th USENIX Security Symposium*, 2018.

[35] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, NIST, 2023.

[36] National Institute of Standards and Technology. Artificial intelligence risk management framework: Generative artificial intelligence profile. NIST AI 600-1, NIST, 2024.

[37] International Organization for Standardization. ISO/IEC 42001:2023 — information technology — artificial intelligence — management system. `https://www.iso.org/standard/81230.html`, 2023.

[38] International Organization for Standardization. ISO/IEC 23894:2023 — artificial intelligence — guidance on risk management. `https://www.iso.org/standard/77304.html`, 2023.

[39] AICPA. Trust services criteria (2017, with 2022 revised points of focus). `https://www.aicpa-cima.com/resources/download/trust-services-criteria`, 2022.

[40] European Union. Regulation (EU) 2024/1689 — artificial intelligence act. https://eur-lex.europa.eu/eli/reg/2024/1689/oj, 2024. Official Journal of the European Union, 12 July 2024.

[41] OWASP Foundation. Artificial intelligence security verification standard (AISVS), v1.0. https://github.com/OWASP/AISVS, 2026.

[42] MITRE Corporation. MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems. https://atlas.mitre.org/, 2026.

[43] Task Force for AI Agents in Healthcare. HAARF: A unified, lifecycle-centric regulatory framework for autonomous AI agents in medical environments. https://github.com/Task-force-for-AI-agents-in-Healthcare/haarf, 2026. Coverage-mapping methodology (EM/PM/NM rubric, coding sheets, reproducible coverage computation).

[44] World Wide Web Consortium. Decentralized identifiers (DIDs) v1.0. https://www.w3.org/TR/did-core/, 2022.

[45] World Wide Web Consortium. Verifiable credentials data model v2.0. https://www.w3.org/TR/vc-data-model-2.0/, 2025.

[46] OpenSSF. SLSA: Supply-chain levels for software artifacts. https://slsa.dev/, 2026.

[47] Coalition for Content Provenance and Authenticity. C2PA technical specification. https://c2pa.org/, 2026.

[48] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, Internet Engineering Task Force, 1997.

[49] Ken Huang. MAESTRO framework: Navigating risks in multi-agent AI systems. https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro, 2025. Cloud Security Alliance.

[50] Dorothy E. Denning and Peter J. Denning. Certification of programs for secure information flow. *Communications of the ACM*, 20(7):504–513, 1977.

[51] Andrew C. Myers and Barbara Liskov. A decentralized model for information flow control. In *16th ACM Symposium on Operating Systems Principles (SOSP)*, 1997.

[52] Andrew C. Myers. JFlow: Practical mostly-static information flow control. In *26th ACM Symposium on Principles of Programming Languages (POPL)*, 1999.

[53] Andrei Sabelfeld and Andrew C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003.

[54] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. Making information flow explicit in HiStar. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.

[55] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. Information flow control for standard OS abstractions. In *21st ACM Symposium on Operating Systems Principles (SOSP)*, 2007.

[56] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[57] Henry M. Levy. *Capability-Based Computer Systems*. Digital Press, 1984.

[58] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.

[59] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960.

[60] ISO/IEC JTC 1. SC 42: Artificial intelligence — subcommittee overview. `https://jtc1info.org/sd-2-history/jtc1-subcommittees/sc-42/`, 2026.

[61] Nemko Digital. ISO/IEC JTC 1/SC 42: Leading AI standards for 2025. `https://digital.nemko.com/standards/iso-iec-jtc-1sc-42`, 2025.

[62] LexAI Europe. ISO/IEC standards on AI. `https://www.lexai-europe.com/1901/standardisation/iso-iec-ai`, 2025.

[63] International Electrotechnical Commission. Coordinated standardization for AI and data security in the era of generative AI: Strategic proposals. `https://iec.ch/system/files/2025-10/wsd_2025_combinedpdf.pdf`, 2025.

[64] CipherNorth. NIST AI 600-1 and AI RMF: Managing risk in generative AI. `https://www.ciphernorth.com/blog/nist-ai-risk-management-framework-rmf`, 2026.

[65] Modulos. Implement NIST AI risk management framework. `https://www.modulos.ai/nist-ai-rmf/`, 2026.